

Enabling Secure Hierarchical Scheduling and Fault-Tolerant Processing for Post-Quantum IIoT Data in Fog Computing

Phinnawat Yaemsanguan, Rangsimann Sattayarom, Ratchanon Wongwitutai, Somchart Fugkeaw
Sirindhorn International Institute of Technology, Thammasat University, Thailand
6622770764@g.siit.tu.ac.th, 6622771747@g.siit.tu.ac.th, 6622780268@g.siit.tu.ac.th

Abstract—Industrial Internet of Things (IIoT) systems generate massive volumes of latency-sensitive data that increasingly strain cloud and network infrastructures. Although fog computing enables low-latency processing at the edge, existing secure fog-assisted IIoT solutions primarily emphasize cryptographic protection while overlooking the efficient orchestration of security-sensitive workloads across heterogeneous fog resources. This challenge becomes more pronounced when post-quantum cryptography and Trusted Execution Environments (TEEs) introduce additional computational and resource overhead.

In this paper, we propose *Spider*, a secure and scalable hierarchical scheduling framework for post-quantum IIoT processing in fog computing. Spider integrates PUF-based device authentication, Kyber-based key establishment, ChaCha20-Poly1305 data protection, and lattice-based CP-ABE access control to provide end-to-end post-quantum security. The framework introduces hierarchical fog-enclave scheduling that jointly considers node capability, latency, trust, queue occupancy, and EPC availability for efficient workload placement. Spider further supports enclave-parallel execution, cryptographic-context reuse, secure workload delegation, and group-based failure recovery using authenticated continuation states. Experimental results show that Spider reduces task-completion latency by up to 35% and recovery latency by up to 63% compared with representative approaches. These results demonstrate that Spider provides secure, resilient, and scalable post-quantum processing for fog-assisted IIoT environments.

Index Terms—Industrial Internet of Things (IIoT), Fog Computing, Post-Quantum Cryptography (PQC), Lattice-Based CP-ABE, Dynamic Load Balancing, Trusted Execution Environment (TEE), Physical Unclonable Function (PUF), Fine-Grained Access Control.

I. INTRODUCTION

Industrial Internet of Things (IIoT) environments comprise large numbers of sensors, machines, and controllers that continuously generate high-volume data streams. As industrial deployments scale, transmitting raw data directly to centralized cloud infrastructures introduces significant communication overhead, network congestion, and processing latency. Fog computing mitigates these limitations by moving computation and storage closer to data sources, enabling low-latency processing and more efficient utilization of network resources.

Despite these advantages, securing fog-assisted IIoT systems remains challenging. Industrial data streams must be authenticated, encrypted, aggregated, and accessed under strict real-time constraints. Existing fog-assisted security frameworks primarily focus on cryptographic protection and fine-

grained access control, while giving limited attention to the execution overhead introduced by security mechanisms themselves. In practice, security operations often become a major performance bottleneck, particularly when large numbers of IIoT devices simultaneously generate data requiring authentication, encryption, and policy enforcement.

The challenge becomes more pronounced in the post-quantum era. Recent studies have demonstrated the feasibility of deploying lattice-based cryptography, such as CRYSTALS-Kyber and lattice-based attribute-based encryption, in IoT and fog environments [?], [?], [?], [?]. However, post-quantum cryptographic primitives generally impose substantially higher computational and memory costs than conventional public-key mechanisms. When applied to large-scale IIoT systems, these costs can increase processing latency, reduce throughput, and overload resource-constrained fog infrastructures. Existing research has largely focused on cryptographic construction and protocol design, while the problem of efficiently executing post-quantum security workloads across distributed fog resources remains insufficiently explored.

Furthermore, modern fog infrastructures increasingly employ Trusted Execution Environments (TEEs) to protect sensitive cryptographic operations and confidential data processing. Although TEEs provide strong isolation guarantees, they introduce unique resource-management challenges. Enclave execution is constrained by limited EPC memory, enclave paging overhead, and secure-context switching costs. Contemporary fog nodes may host multiple concurrent enclaves, enabling parallel secure execution but simultaneously creating contention among enclave resources. As workload intensity increases, improper placement of security-sensitive tasks may lead to EPC saturation, queue buildup, enclave contention, and severe performance degradation.

Existing fog load-balancing approaches [?], [?], [?], [?] primarily optimize latency, energy consumption, or computational utilization under conventional execution environments. These approaches generally treat each fog node as a single scheduling entity and do not explicitly consider enclave-level resource constraints, post-quantum cryptographic overhead, execution trustworthiness, or secure workload locality. Consequently, they are ill-suited for coordinating security-intensive workloads in heterogeneous TEE-enabled fog infrastructures.

Therefore, a critical research gap exists in the design of

an integrated scheduling framework capable of jointly managing post-quantum security workloads, enclave resources, and heterogeneous fog-node capabilities. Such a framework must dynamically balance workloads across fog nodes while simultaneously orchestrating execution among multiple enclaves within each node. It must also account for EPC availability, workload urgency, trust conditions, and runtime resource fluctuations to maintain both security and performance under dynamic IIoT conditions.

To address these challenges, we propose **Spider** (Secure Priority-aware Intelligent Dual-level Enclave Routing), a hierarchical enclave-aware scheduling framework for post-quantum secure fog-assisted IIoT systems. Spider integrates a post-quantum security architecture based on PUF-rooted authentication, CRYSTALS-Kyber key establishment, ChaCha20-Poly1305 data protection, and split-phase lattice-based CP-ABE access control. More importantly, Spider introduces a two-level scheduling mechanism that jointly performs inter-node workload orchestration across heterogeneous fog nodes and intra-node scheduling across multiple parallel enclaves. Through capability-aware node selection, EPC-aware scheduling, computation-reuse optimization, enclave-parallel execution, and secure workload recovery, Spider mitigates enclave contention and improves secure-processing scalability for large-scale IIoT environments.

The main contributions of this work are summarized as follows:

- 1) **Spider: Hierarchical Enclave-Aware Scheduling for Secure Fog Computing**

We propose **Spider**, a hierarchical scheduling framework that jointly performs stability-aware Master Fog Node (MFN) coordination, capability-aware fog-node selection, and EPC-aware enclave scheduling. Unlike conventional fog schedulers that treat fog nodes as single execution units, Spider explicitly incorporates enclave-level resource dynamics, runtime trust, workload urgency, execution locality, and secure delegation to optimize post-quantum workload placement across heterogeneous fog infrastructures.

- 2) **Post-Quantum TEE-Assisted Secure Processing Architecture**

We design an end-to-end fog-assisted IIoT security architecture integrating PUF-rooted device authentication, CRYSTALS-Kyber key establishment, ChaCha20-Poly1305 data protection, and split-phase lattice-based CP-ABE access control. Security-sensitive cryptographic operations are confined to TEEs, while non-sensitive processing is safely offloaded to REEs, enabling scalable and quantum-resistant secure data processing.

- 3) **Secure Micro-Batch Aggregation and Enclave-Parallel Execution**

We introduce a gateway-assisted validation and deterministic micro-batch processing mechanism that supports authenticated packet filtering, integrity-protected batch formation, enclave-parallel workload decomposi-

tion, and deterministic aggregation verification. The proposed design reduces repeated cryptographic verification overhead while enabling scalable secure processing of high-volume IIoT streams.

- 4) **Decentralized Fault-Tolerant Recovery with Authenticated Continuation States**

We develop a decentralized recovery framework based on collaborative failure detection, quorum-based failure confirmation, and authenticated continuation states. By recovering only unfinished enclave-parallel workloads rather than entire execution contexts, Spider minimizes recomputation overhead while preserving execution continuity, integrity, and recovery authenticity under fog-node failures.

II. RELATED WORK

A. Fog-Assisted IIoT Security and TEE-Based Execution

Fog computing has become an important paradigm for enabling low-latency processing and distributed security services in Industrial Internet of Things (IIoT) environments. Lightweight authentication is commonly achieved through Physical Unclonable Function (PUF)-based mechanisms. Representative schemes [1], [2] support efficient device authentication and group verification while reducing secret-key storage requirements. However, these approaches primarily focus on authentication and message integrity without addressing secure data sharing or fine-grained access control.

To protect outsourced data, Ciphertext-Policy Attribute-Based Encryption (CP-ABE) has been widely adopted. Existing studies [3], [5] provide fine-grained access control and revocation, while [4] further introduces a lattice-based quantum-resistant CP-ABE construction. Although these schemes improve confidentiality and policy enforcement, they generally operate independently of lightweight authentication mechanisms and do not consider runtime execution efficiency.

Trusted Execution Environments (TEEs) have been explored to protect sensitive computation at the edge and fog layers. Prior studies [6], [7] highlight practical challenges including enclave-memory limitations, performance overhead, and susceptibility to side-channel attacks. More comprehensive fog-security frameworks [8]–[10] incorporate trust management, authentication, and access control, but do not jointly address post-quantum protection, TEE-assisted execution, and runtime workload orchestration. Consequently, existing solutions provide isolated security mechanisms rather than an integrated framework for secure and efficient fog-assisted IIoT processing.

B. Post-Quantum Cryptography for IIoT Systems

The emergence of quantum computing has motivated the adoption of post-quantum cryptography (PQC) for resource-constrained IoT and IIoT environments. Lattice-based schemes, particularly CRYSTALS-Kyber and CRYSTALS-Dilithium, have received significant attention due to their

TABLE I
COMPREHENSIVE FEATURE COMPARISON OF RELATED WORKS AND SPIDER

Scheme	PUF	PQC	Fine-Grained AC	Revocation	TEE Support	Fog/Edge	Load Balancing
[4]	×	✓	✓	✓	×	✓	×
[5]	×	×	✓	✓	×	✓	×
[22]	×	×	×	×	×	✓	✓
[34]	×	✓	×	×	✓	×	×
[35]	×	✓	×	×	×	×	✓
[36]	×	✓	×	×	×	×	×
[37]	×	×	×	×	×	✓	✓
[39]	×	×	×	×	×	✓	✓
[40]	×	×	×	×	×	✓	✓
(Ours)	✓	✓	✓	✓	✓	✓	✓

strong security foundations and practical performance. Existing studies [12], [13] investigate their integration into communication protocols and network stacks, while [14], [15] evaluate their deployment in IoT and IIoT environments with respect to execution overhead, bandwidth consumption, and energy efficiency.

Several works further explore hardware-assisted protection. For example, [16], [17] combine PUF-based mechanisms with post-quantum primitives to strengthen device authentication and reduce reliance on stored secrets. Broader investigations [18], [19] discuss deployment challenges such as cryptography, interoperability, and scalability in edge-fog ecosystems.

Despite these advances, existing research primarily focuses on cryptographic construction, protocol integration, or performance benchmarking. Limited attention has been given to how post-quantum security workloads should be efficiently executed and scheduled across heterogeneous fog infrastructures, particularly when TEE-based secure execution is involved.

C. Load Balancing in Heterogeneous Fog Environments

Efficient workload scheduling is essential in fog-assisted IIoT systems because fog nodes differ significantly in processing capability, memory resources, and network conditions. Existing approaches [22], [24], [25] dynamically allocate workloads according to latency, energy consumption, or resource utilization, while multi-objective optimization techniques [26] jointly optimize delay, cost, and energy efficiency.

More recent studies have considered the interaction between security and scheduling. For example, [27], [28] introduce secure or energy-aware load-balancing mechanisms for fog environments. Meanwhile, [29], [30] demonstrate that trusted execution environments introduce additional runtime overhead due to enclave transitions, memory constraints, and cryptographic processing. These findings indicate that security mechanisms can significantly influence scheduling performance.

Table I summarizes representative studies. Existing security-oriented schemes [4], [5] provide fine-grained access control and revocation but do not consider runtime workload management. Conversely, fog load-balancing approaches [22], [37], [39], [40] improve resource utilization and latency but operate under conventional execution models and do not account

for post-quantum cryptographic overhead, enclave contention, EPC pressure, or trust-aware scheduling decisions.

Overall, current research treats security and scheduling as largely independent problems. To the best of our knowledge, no existing framework jointly integrates lightweight authentication, post-quantum access control, TEE-assisted secure execution, and hierarchical enclave-aware scheduling for heterogeneous fog-assisted IIoT environments. This gap motivates the design of Spider, which explicitly considers both security overhead and runtime scheduling efficiency when orchestrating secure workloads across distributed fog resources.

III. OUR PROPOSED SYSTEM

This section presents Spider, a post-quantum secure fog-assisted architecture for IIoT environments. Spider combines lightweight device authentication, post-quantum cryptographic protection, TEE-assisted secure processing, and hierarchical workload scheduling to enable scalable and secure execution of security-intensive workloads across heterogeneous fog infrastructures.

A. System Model

As illustrated in Fig. 1, Spider adopts a four-layer architecture consisting of the *Edge Layer*, *Fog Layer*, *Cloud Layer*, and *Secure Layer*. The central component of the architecture is the Spider Scheduler, which performs hierarchical workload orchestration across distributed fog nodes and multiple TEE enclaves. The architecture consists of the following entities.

- 1) **Attribute Authority (AA)** serves as the trusted security authority responsible for initializing the lattice-based CP-ABE system, generating public parameters, issuing attribute secret keys, and defining access-control policies for authorized users.
- 2) **IIoT Devices** are resource-constrained sensors and industrial controllers equipped with Physical Unclonable Functions (PUFs). Each device performs lightweight authentication and authenticated encryption before transmitting data toward the fog infrastructure.
- 3) **Edge Gateway (GW)** serves as the first aggregation point between IIoT devices and the fog infrastructure. It authenticates incoming device transmissions, filters

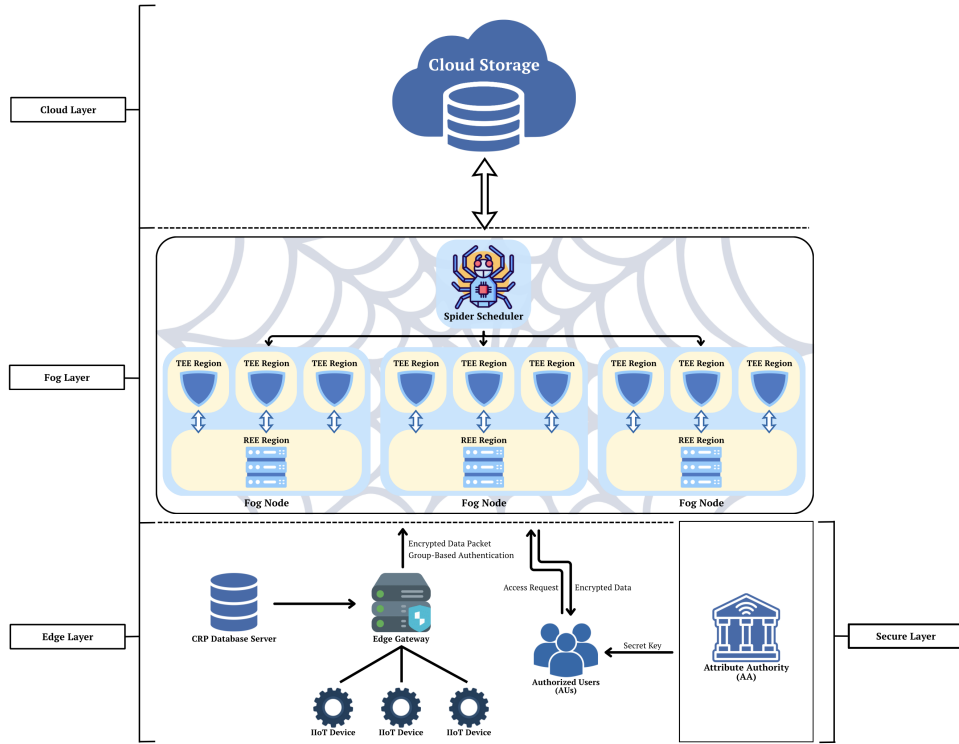


Fig. 1. Spider-Enabled Fog Computing Architecture for Secure IIoT Data Processing.

invalid or malicious traffic, performs integrity verification, and aggregates validated packets into micro-batches before forwarding them to the fog layer for secure processing.

- 4) **Master Fog Node (MFN)** acts as the global scheduling coordinator of Spider. It continuously collects runtime telemetry from participating fog nodes and executes the Spider scheduling algorithm. Based on workload characteristics, enclave availability, resource utilization, and trust conditions, the MFN determines the most suitable fog node and enclave for processing each workload.
- 5) **Fog Nodes (FNs)** are TEE-enabled computing servers responsible for executing security-sensitive workloads. Each fog node hosts multiple enclaves that enable parallel secure execution of cryptographic and data-processing tasks. By leveraging multiple enclaves, fog nodes can improve processing throughput while maintaining strong isolation guarantees for sensitive operations.

Each fog node follows a dual-execution architecture:

- **TEE Region:** Secure execution of cryptographic operations, key management, aggregation, and sensitive CP-ABE processing.
- **REE Region:** Execution of non-sensitive tasks such as policy expansion, metadata management, and auxiliary processing.

- 6) **Spider Scheduler** performs hierarchical scheduling at two levels:

- **Inter-node Scheduling**, which selects the most suitable fog node according to workload demand, node capability, EPC availability, latency, and trust condition.
- **Intra-node Scheduling**, which distributes workloads among multiple enclaves within the selected fog node to minimize enclave contention and improve secure-processing throughput.

To further improve scalability, Spider supports workload decomposition, computation reuse, and secure delegation when local resources become unavailable.

- 7) **Cloud Storage** encrypted IIoT data, encrypted aggregation results, and associated metadata. The cloud is assumed to be honest-but-curious and therefore cannot access plaintext information.
- 8) **Authorized Users (AUs)** retrieve encrypted data from the cloud and perform decryption only when their attribute sets satisfy the embedded CP-ABE access policy.

B. Threat Model

We consider a probabilistic polynomial-time (PPT) adversary with full control over public communication channels in the IIoT environment. The adversary can eavesdrop, intercept, modify, inject, replay, delay, or drop messages exchanged among IIoT devices, gateways, fog nodes, and cloud services. In addition, the adversary may compromise software components outside trusted execution boundaries, manipulate scheduling telemetry, inject malicious workloads, or attempt unauthorized access to encrypted cloud data.

TABLE II
MAIN NOTATIONS

Notation	Description
λ	Security parameter
q	Modulus of lattice cryptosystem
Ω	Attribute universe
ID_j	Hashed attribute identifier
MPK, MSK	CP-ABE public and master secret keys
SK_u	User attribute secret key
pk_{GW}, sk_{GW}	Gateway Kyber public/secret key pair
pk_{FN}, sk_{FN}	Fog-node Kyber public/secret key pair
B_k	Validated workload batch
$\Phi(B_k)$	Spider workload profile
F_j	Fog node
$E_{j,k}$	Enclave hosted by fog node F_j
$\Psi(F_j)$	Runtime state of fog node F_j
M_{EPC}	Available enclave page cache (EPC) memory
U_j	Runtime trust score of fog node F_j
C_{policy}	Policy cache repository
C_{crypto}	Cryptographic-context cache repository
$Root_k$	Aggregation commitment
Ω_k	Final outsourced object

The adversary may fully compromise the Rich Execution Environment (REE) of fog nodes, allowing memory inspection and execution manipulation. However, enclave-resident code, memory, and sealed secrets remain protected by the Trusted Execution Environment (TEE). The adversary may also launch integrity attacks against ciphertexts, metadata, aggregation outputs, and scheduling states, as well as exploit timing, cache, or memory-access side channels against TEEs. Furthermore, we consider a quantum-capable adversary that can break conventional public-key cryptography, motivating the use of post-quantum primitives.

Trust Assumptions The Attribute Authority (AA) is fully trusted and correctly generates system parameters, access policies, and attribute secret keys. Cloud storage is assumed to be honest-but-curious, faithfully storing encrypted data while attempting to infer information from ciphertexts and metadata. TEEs are trusted to provide confidentiality and integrity for enclave-resident code and cryptographic secrets, whereas REEs are considered untrusted. Authorized users are assumed to protect their attribute secret keys.

Under these assumptions, Spider aims to ensure confidentiality, integrity, access-control enforcement, scheduling robustness, and secure execution of post-quantum protected workloads in heterogeneous fog-assisted IIoT environments.

C. System Process

The proposed scheme operates through seven phases: system initialization, hardware-bound IIoT encryption, gateway validation and micro-batch formation, capability-aware leader election, dynamic load balancing, TEE-assisted split-phase CP-ABE processing, and authorized user decryption. Table II presents list of notations used in all system phases.

Phase I: System Initialization: This phase establishes the security, communication, and scheduling infrastructure required by Spider. The objective is to initialize post-quantum

access control, secure communication channels, and the runtime environment for enclave-aware workload orchestration.

Step 1: Security Infrastructure Initialization: The Attribute Authority (AA) initializes the lattice-based CP-ABE system by defining the attribute universe Ω and generating the public parameters and master secret key:

$$(MPK, MSK) \leftarrow \text{Setup}(1^\lambda, \Omega) \quad (1)$$

To conceal semantic information, each attribute is mapped to a hashed identifier

$$ID_j = \mathcal{H}(\text{attr}_j) \quad (2)$$

Access policies are converted into their LSSS representations and cached for future reuse, while each authorized user receives an attribute-bound secret key

$$SK_u \leftarrow \text{KeyGen}(MSK, \text{Attr}_u) \quad (3)$$

Step 2: Post-Quantum Communication Initialization: The gateway and fog nodes establish post-quantum communication capabilities using CRYSTALS-Kyber:

$$(pk_{GW}, sk_{GW}), (pk_{FN}, sk_{FN}) \leftarrow \text{KeyGen}_{\text{Kyber}}(1^\lambda) \quad (4)$$

During registration, each IIoT device establishes a shared session key

$$(CT_{KEM}, K_S) \leftarrow \text{Encaps}(pk_{GW}) \quad (5)$$

which is subsequently used to derive communication keys.

To support efficient post-quantum operations, fog nodes cache reusable lattice structures

$$\hat{A} = \text{NTT}(\text{ExpandA}(\text{seed}_{FN})) \quad (6)$$

Step 3: Spider Infrastructure Initialization: Each fog node initializes multiple TEE enclaves

$$F_j = \{E_{j,1}, \dots, E_{j,m_j}\} \quad (7)$$

and deploys telemetry monitors to track queue occupancy, service rate, and EPC availability:

$$\Psi(F_j) = (Q_j, \mu_j, M_{EPC}) \quad (8)$$

An initial trust value

$$U_j^{(0)} = 1 \quad (9)$$

is assigned to each fog node and updated dynamically during runtime.

The resulting infrastructure provides the runtime visibility, reusable cryptographic contexts, and enclave resources required by Spider to perform hierarchical workload placement and secure parallel execution in subsequent phases.

Phase II: Hardware-Bound IIoT Protection: This phase establishes device-specific protection for sensed data before it enters the fog infrastructure. Spider combines hardware-rooted identity, post-quantum key establishment, and lightweight authenticated encryption to generate integrity-protected packets suitable for high-throughput IIoT environments.

Step 1: Hardware-Bound Session Key Establishment: During device registration or session initialization, device ID_i derives a hardware-rooted secret from its Physical Unclonable Function (PUF). A noisy response is first generated and subsequently reconstructed into a stable secret:

$$R_{noisy,i} \leftarrow f_{PUF}(C_i) \quad (10)$$

$$R_{secret,i} \leftarrow \text{FuzzyRep}(R_{noisy,i}, HD_i) \quad (11)$$

The device then establishes a post-quantum shared secret with the gateway through Kyber encapsulation:

$$(CT_{KEM,i}, K_{root,i}) \leftarrow \text{Encaps}(pk_{GW}) \quad (12)$$

The resulting root key is cryptographically bound to the physical device instance through

$$K_{base,i} = \text{KDF}(K_{root,i} \parallel R_{secret,i} \parallel ID_i) \quad (13)$$

thereby preventing key transfer across cloned devices.

Step 2: Per-Packet Protection and Authentication: For each sensed data item m_i , the device derives a fresh packet key

$$K_{S,i} = \text{KDF}(K_{base,i} \parallel t_i \parallel ctr_i) \quad (14)$$

where t_i and ctr_i denote the packet timestamp and counter, respectively.

The associated authenticated data is constructed as

$$AAD_i = ID_i \parallel t_i \parallel ctr_i \quad (15)$$

The sensed data is encrypted using ChaCha20-Poly1305:

$$(CT_i, Tag_i) \leftarrow \text{Enc}(K_{S,i}, N_i, m_i, AAD_i) \quad (16)$$

To support efficient packet validation, the device derives an authentication key

$$K_{Auth,i} = \text{KDF}(K_{S,i}, \text{"auth"}) \quad (17)$$

and computes

$$Auth_i = \text{HMAC}(K_{Auth,i}, ID_i \parallel CT_i \parallel Tag_i \parallel t_i \parallel ctr_i) \quad (18)$$

Step 3: Secure Packet Formation: The protected packet is constructed as

$$P_i = \{ID_i, CT_i, N_i, AAD_i, Tag_i, Auth_i, t_i, ctr_i\} \quad (19)$$

The packet is then transmitted to the gateway for validation, micro-batch formation, and workload generation in Phase III.

Phase III: Gateway Validation and Workload Formation: This phase performs packet validation, malicious-traffic filtering, and micro-batch construction before workload submission to Spider. By eliminating invalid packets at the gateway, the system prevents unnecessary cryptographic processing at the fog layer and improves the efficiency of subsequent scheduling decisions.

Step 1: Packet Validation: For each received packet P_i , the gateway reconstructs the packet authentication key and verifies its integrity:

$$K_{Auth,i} = \text{KDF}(K_{S,i}, \text{"auth"}) \quad (20)$$

$$Auth'_i = \text{HMAC}(K_{Auth,i}, ID_i \parallel CT_i \parallel Tag_i \parallel t_i \parallel ctr_i) \quad (21)$$

A packet is accepted only if

$$Auth'_i = Auth_i \quad (22)$$

Packets failing verification are discarded immediately, preventing invalid or replayed traffic from entering the fog infrastructure.

Step 2: Micro-Batch Formation: All validated packets collected within the aggregation window t_{window} are grouped into a micro-batch

$$\mathcal{P}_{valid} = \{P_i \mid Auth'_i = Auth_i\} \quad (23)$$

To ensure batch-level integrity, the gateway computes

$$GTag = H(\text{Sort}(\{Auth_i \mid P_i \in \mathcal{P}_{valid}\}) \parallel t_G) \quad (24)$$

where t_G denotes the gateway timestamp.

The resulting batch is

$$B_k = (BID, \mathcal{P}_{valid}, GTag, t_G) \quad (25)$$

Step 3: Workload Descriptor Generation: To support Spider scheduling, the gateway derives a lightweight workload descriptor from the validated batch:

$$\Phi(B_k) = (S_k, \omega_k, \eta_k, \delta_k) \quad (26)$$

where S_k denotes valid batch size, ω_k denotes estimated workload intensity, η_k denotes security urgency, and δ_k denotes deadline sensitivity.

The workload intensity is estimated as

$$\omega_k = \alpha_1 S_k + \alpha_2 N_{attr} + \alpha_3 D_{policy} \quad (27)$$

where N_{attr} denotes the expected attribute-processing complexity and D_{policy} denotes the policy depth associated with the target workload.

The pair

$$(B_k, \Phi(B_k)) \quad (28)$$

is forwarded to the Master Fog Node (MFN) for capability-aware and enclave-aware scheduling in Phase IV.

The complete validation and micro-batch formation procedure is summarized in Algorithm 1.

Algorithm 1 Gateway Validation and Workload Formation**Require:** Packet stream within window t_{window} **Ensure:** Validated batch B_k and workload profile $\Phi(B_k)$

```

1:  $\mathcal{P}_{valid} \leftarrow \emptyset, \mathcal{M} \leftarrow \emptyset, N_{drop} \leftarrow 0$ 
2: for all  $P_i$  in  $t_{window}$  do
3:    $K_{Auth,i} \leftarrow \text{KDF}(K_{S,i}, \text{"auth"})$ 
4:    $Auth'_i \leftarrow \text{HMAC}(K_{Auth,i}, ID_i \parallel CT_i \parallel Tag_i \parallel t_i \parallel$ 
      $ctr_i)$ 
5:   if  $Auth'_i = Auth_i$  and  $(t_i, ctr_i)$  is fresh then
6:      $\mathcal{P}_{valid} \leftarrow \mathcal{P}_{valid} \cup \{P_i\}$ 
7:      $\mathcal{M} \leftarrow \mathcal{M} \cup \{Auth_i\}$ 
8:   else
9:      $N_{drop} \leftarrow N_{drop} + 1$ 
10:  end if
11: end for
12: if  $\mathcal{P}_{valid} = \emptyset$  then
13:   return  $\emptyset$ 
14: end if
15:  $t_G \leftarrow \text{GatewayTimestamp}()$ 
16:  $BID \leftarrow H(ID_{GW} \parallel t_G \parallel |\mathcal{P}_{valid}|)$ 
17:  $GTag \leftarrow H(\text{Sort}(\mathcal{M}) \parallel t_G \parallel BID)$ 
18:  $B_k \leftarrow (BID, \mathcal{P}_{valid}, GTag, t_G)$ 
19:  $S_k \leftarrow |\mathcal{P}_{valid}|$ 
20:  $\omega_k \leftarrow \alpha_1 S_k + \alpha_2 N_{attr} + \alpha_3 D_{policy}$ 
21:  $\eta_k \leftarrow \gamma_1 \frac{N_{drop}}{S_k + N_{drop}} + \gamma_2 \text{AnomalyFlag}(B_k)$ 
22:  $\delta_k \leftarrow \text{DeadlineScore}(B_k)$ 
23:  $\Phi(B_k) \leftarrow (S_k, \omega_k, \eta_k, \delta_k)$ 
24: return  $(B_k, \Phi(B_k))$ 

```

Phase IV: Spider Hierarchical Enclave-Aware Scheduling:

Spider (*Secure Priority-Aware Intelligent Dual-Level Enclave Routing*) is a hierarchical scheduling framework designed for secure workload orchestration in heterogeneous fog-assisted IIoT environments. Unlike conventional fog schedulers that optimize only node-level resource utilization, Spider addresses a security-aware workload placement problem in which each incoming workload must be assigned to both a fog node and a specific enclave. The objective is to minimize processing latency and enclave contention while satisfying trust, memory, and deadline requirements.

Spider is motivated by two observations. First, post-quantum cryptographic workloads introduce substantial computational overhead when executed on resource-constrained fog infrastructures. Second, Trusted Execution Environments (TEEs) suffer from limited Enclave Page Cache (EPC) memory, causing severe performance degradation when enclave contention or paging occurs. Therefore, Spider jointly optimizes workload placement, EPC utilization, execution locality, and scheduling stability through a hierarchical scheduling process that spans both fog-node and enclave levels.

Step 1: Stability-Aware Master Fog Node Election:

Spider first elects a Master Fog Node (MFN) responsible for global workload coordination. For each fog node F_j , Spider

computes

$$Score_{MFN}(F_j) = \alpha_1 C_j + \alpha_2 M_j - \alpha_3 L_j + \alpha_4 T_j + \alpha_5 R_j \quad (29)$$

where C_j denotes computational capability, M_j denotes available secure-execution resources, L_j denotes communication latency, T_j denotes runtime trustworthiness, and R_j denotes node readiness.

The readiness score is computed as

$$R_j = 1 - (\beta_1 \bar{Q}_j + \beta_2 \rho_{epc}(F_j) + \beta_3 \rho_{ree}(F_j)) \quad (30)$$

where $\bar{Q}_j$ denotes normalized enclave queue occupancy, $\rho_{epc}(F_j)$ denotes EPC utilization, and $\rho_{ree}(F_j)$ denotes REE backlog.

To prevent leadership oscillation, Spider elects

$$MFN = \arg \max_{F_j} (Score_{MFN}(F_j) - \gamma \Delta Score(F_j)), \quad (31)$$

where $\Delta Score(F_j)$ captures short-term score variations. Only nodes satisfying

$$R_j \geq \tau \quad (32)$$

are eligible for election.

The elected MFN continuously coordinates workload scheduling, resource monitoring, enclave assignment, and recovery operations.

Step 2: Runtime Monitoring and Workload Profiling:

The MFN continuously collects runtime telemetry from participating fog nodes, including queue occupancy, service rate, EPC availability, computational capability, network latency, and trust status.

For each validated batch B_k , Spider constructs

$$\Phi(B_k) = (S_k, \omega_k, \eta_k, \delta_k) \quad (33)$$

where S_k denotes batch size, ω_k denotes workload intensity, η_k denotes security urgency, and δ_k denotes deadline sensitivity.

This workload profile enables Spider to distinguish lightweight tasks from security-intensive post-quantum workloads before scheduling.

Step 3: Capability- and Trust-Aware Fog-Node Selection:

Spider evaluates candidate fog nodes according to waiting time, communication latency, EPC pressure, computational capability, runtime trust, and cryptographic-context reuse:

$$Score_F(F_j, B_k) = w_1 T_{wait} + w_2 L_j + w_3 P_{epc} + w_4 P_{cap} + w_5 P_{trust} - w_6 R_{reuse} \quad (34)$$

The destination fog node is selected as

$$F^* = \arg \min_{F_j} Score_F(F_j, B_k) \quad (35)$$

Unlike conventional schedulers that primarily consider resource availability, Spider explicitly incorporates EPC pressure, trust status, and execution-locality reuse.

Step 4: EPC-Aware Enclave Selection: After selecting F^* , Spider performs enclave-level scheduling among the TEEs hosted by the selected fog node.

For each enclave $E_{j,k}$

$$Score_E(E_{j,k}, B_k) = z_1 T_{wait} + z_2 P_{epc} - z_3 A \quad (36)$$

where T_{wait} denotes expected waiting time, P_{epc} denotes EPC pressure, and A denotes workload affinity.

The selected enclave is

$$E^* = \arg \min_{E_{j,k} \in F^*} Score_E(E_{j,k}, B_k) \quad (37)$$

This dual-level scheduling process simultaneously captures fog-node heterogeneity and enclave-level execution dynamics.

Step 5: Adaptive Enclave-Parallel Batch Decomposition: To exploit enclave-level parallelism, Spider dynamically partitions security-intensive workloads into enclave-independent sub-batches:

$$B_k \rightarrow \{B_k^{(1)}, \dots, B_k^{(r)}\} \quad (38)$$

where the decomposition factor is determined according to workload intensity, enclave availability, queue conditions, and EPC capacity.

Each sub-batch is independently assigned to an available enclave and processed concurrently.

Step 6: Stability-Aware Scheduling Control: To avoid scheduling oscillation under bursty workloads, Spider applies exponential score smoothing:

$$\bar{S}_j(t) = \alpha_s Score_F(F_j, B_k) + (1 - \alpha_s) \bar{S}_j(t-1) \quad (39)$$

Scheduling updates are performed only when score variations exceed a predefined threshold. This mechanism reduces unnecessary workload migration and improves long-term scheduling stability.

Step 7: Secure Delegation and Recovery Preparation: If no feasible enclave is available or the selected fog node becomes overloaded, Spider initiates secure delegation using a lightweight continuation state

$$\Xi_k = (ID_k, SubID_k, Prog_k, \Phi(B_k), A_k, epoch_k, T_k) \quad (40)$$

where $Prog_k$ denotes execution progress and A_k denotes the associated policy reference.

The remaining workload is estimated as

$$\omega_k^{rem} = (1 - Prog_k) \omega_k \quad (41)$$

The continuation state is authenticated by

$$\sigma_k = Sign_{SK_{FN}}(H(\Xi_k)) \quad (42)$$

and protected through Kyber-based encapsulation:

$$(K_s, CT_{KEM}) \leftarrow \text{Encaps}(pk_{FN_r}) \quad (43)$$

$$CT_{del} = \text{Enc}_{K_s}(\Xi_k \parallel \sigma_k) \quad (44)$$

The delegated state is accepted only if

$$Verify_{PK_{FN}}(\sigma_k, H(\Xi_k)) = 1 \quad (45)$$

and

$$epoch_k = epoch_{current} \quad \wedge \quad T_k < T_{exp} \quad (46)$$

To prioritize recovery operations, Spider computes

$$R_k = \lambda_1 \omega_k^{rem} + \lambda_2 \eta_k + \lambda_3 \delta_k \quad (47)$$

where larger values indicate higher recovery priority.

Through stability-aware MFN election, capability-aware fog-node selection, EPC-aware enclave scheduling, adaptive workload decomposition, and secure delegation, Spider provides a unified framework for scalable and secure orchestration of post-quantum workloads in heterogeneous fog-assisted IIoT environments.

Phase V: Spider-Enabled Multi-Enclave Secure Processing: After Spider determines the destination fog node and enclave assignment in Phase IV, the workload enters the secure execution stage. In this phase, we employ *enclave-parallel secure processing*, where multiple enclaves cooperatively process independent workload partitions. This design reduces EPC contention, improves secure-processing throughput, and prevents a single enclave from becoming a performance bottleneck under high-volume post-quantum IIoT workloads.

Step 1: Secure Batch Reception and Parallel Partitioning: Upon receiving the validated batch B_k , the fog node first verifies the gateway integrity commitment. Let

$$GTag' = H(BID \parallel epoch_k \parallel B_k) \quad (48)$$

where $GTag'$ denotes the recomputed batch tag. The batch is accepted only if

$$GTag' = GTag \quad (49)$$

Based on the scheduling decision generated in Phase IV, Spider either processes the batch directly or partitions it into enclave-independent sub-batches

$$B_k \rightarrow \{B_k^{(1)}, \dots, B_k^{(r)}\} \quad (50)$$

where the decomposition factor is determined as

$$r = \min \left(N_E, \left\lceil \frac{\omega_k}{\Gamma_E} \right\rceil \right) \quad (51)$$

with N_E denoting the number of available enclaves and Γ_E representing the maximum sustainable workload per enclave.

Spider guarantees exact-once execution through

$$\bigcup_{i=1}^r B_k^{(i)} = B_k \quad (52)$$

and

$$B_k^{(i)} \cap B_k^{(j)} = \emptyset, \quad i \neq j \quad (53)$$

Each sub-batch is assigned

$$SubID_i = H(BID \parallel i \parallel epoch_k) \quad (54)$$

and dispatched to enclave $E^{*}(i)$

Step 2: EPC-Aware Enclave-Parallel Aggregation: Each enclave independently processes its assigned sub-batch. To avoid EPC overflow, Spider adopts streaming aggregation rather than materializing the entire workload inside enclave memory. The enclave memory footprint is constrained by

$$Mem(E_i) \leq \eta M_{EPC} \quad (55)$$

where η is a safety factor and M_{EPC} denotes available EPC capacity.

The aggregation state is updated incrementally as

$$S_j = H(S_{j-1} \parallel m_j) \quad (56)$$

where m_j denotes the decrypted packet payload.

A master aggregation key is generated within the enclave and used to derive enclave-specific encryption keys

$$K_{chunk}^{(i)} = \text{KDF}(K_{master}, BID, SubID_i, epoch_k) \quad (57)$$

The local aggregation output is encrypted as

$$C_i = \text{Enc}_{K_{chunk}^{(i)}}(\mathcal{M}_i) \quad (58)$$

and bound to its execution context through

$$h_i = H(BID \parallel SubID_i \parallel C_i \parallel epoch_k) \quad (59)$$

To exploit execution locality, Spider reuses cached policy structures whenever

$$\text{Sim}(\Phi(B_k), \Phi(B_x)) \geq \tau_{reuse} \quad (60)$$

where τ_{reuse} denotes the reuse threshold and $\Phi(\cdot)$ denotes the workload profile generated in Phase IV.

Step 3: Deterministic Merge and Execution Commitment: After enclave execution completes, Spider applies deterministic merge ordering

$$\mathcal{O} = \text{Sort}(SubID_1, \dots, SubID_r) \quad (61)$$

ensuring identical aggregation results regardless of enclave execution timing.

The encrypted aggregation object becomes

$$CT_{AES} =_{\Gamma_i \in \mathcal{O}} C_i \quad (62)$$

Spider then generates a global execution commitment

$$Root_k = H(BID \parallel epoch_k \parallel h_{(1)} \parallel \dots \parallel h_{(r)}) \quad (63)$$

where $h_{(i)}$ denotes the ordered enclave digest.

The commitment serves as both an integrity anchor and a recovery checkpoint. Any replay, omission, duplication, or reordering attack results in

$$Root'_k \neq Root_k \quad (64)$$

Step 4: Split-Phase Post-Quantum Access Protection: Spider protects the master aggregation key using lattice-based CP-ABE. Inside the enclave, the master ciphertext component is generated as

$$CT_0 = \text{Enc}_{ABE}(K_{master}, \mathcal{P}) \quad (65)$$

where $\mathcal{P}$ denotes the access policy.

To reduce enclave workload, Spider exports only blinded policy-expansion components

$$\Psi_{out} = (CT_0, V_{base}, TID) \quad (66)$$

to the REE.

The REE performs policy expansion without accessing

$$K_{master}, s, \mathbf{v} \quad (67)$$

or any plaintext-sensitive information. Consequently,

$$\Pr[\text{REE learns } K_{master}] \approx 0 \quad (68)$$

under the underlying lattice-security assumptions.

Step 5: Authenticated Cloud Outsourcing: Finally, Spider signs the encrypted aggregation object together with its execution commitment:

$$\sigma = \text{Sign}_{\text{Dilithium}}(H(CT_{AES} \parallel CT_{ABE} \parallel Root_k)) \quad (69)$$

The final outsourced object is

$$\Omega = (BID, epoch_k, CT_{AES}, CT_{ABE}, Root_k, \sigma) \quad (70)$$

The cloud accepts the upload only if

$$\text{Verify}_{\text{Dilithium}}(pk_{FN}, \sigma) = 1 \quad (71)$$

Through enclave-parallel execution, EPC-aware streaming aggregation, execution-locality reuse, deterministic execution commitments, and split-phase post-quantum access protection, Spider transforms secure fog processing from a single-enclave execution model into a scalable multi-enclave execution framework for large-scale post-quantum IIoT environments.

Phase VI: User Verification and Decryption: An authorized user retrieves the outsourced object

$$\Omega_k = (BID, epoch_k, CT_{AES}, CT_{ABE}, Root_k, \sigma) \quad (72)$$

from cloud storage. The objective of this phase is to verify the authenticity of the outsourced object, recover the protected aggregation key, validate the enclave-parallel execution commitment, and reconstruct the aggregated IIoT data.

Step 1: Object Authenticity Verification: The user first verifies the Dilithium signature generated by the fog node:

$$\text{Verify}_{\text{Dilithium}}(pk_{FN}, \sigma, H(BID \parallel epoch_k \parallel CT_{AES} \parallel CT_{ABE} \parallel Root_k)) = 1 \quad (73)$$

If the verification fails, the object is rejected. This step ensures that the encrypted aggregation object, access-control ciphertext, and execution commitment have not been modified after fog-layer processing.

Step 2: Access-Policy Validation and Key Recovery:

The user checks whether its attribute set $Attr_u$ satisfies the access policy embedded in CT_{ABE} . If the policy is not satisfied, decryption is aborted. Otherwise, the user derives the reconstruction coefficients and recovers the master aggregation key:

$$K_{master} \leftarrow \text{Dec}_{ABE}(CT_{ABE}, SK_u) \quad (74)$$

Only users whose attributes satisfy the CP-ABE policy can recover K_{master}

Step 3: Execution Commitment Verification: Using the chunk metadata associated with CT_{AES} , the user parses the encrypted aggregation into ordered chunks $\{C_1, \dots, C_r\}$. For each chunk, the user recomputes

$$h_i = H(BID \parallel SubID_i \parallel C_i \parallel epoch_k) \quad (75)$$

The user then reconstructs the execution commitment

$$Root'_k = H(BID \parallel epoch_k \parallel h_{(1)} \parallel \dots \parallel h_{(r)}) \quad (76)$$

where $h_{(i)}$ follows the deterministic ordering defined in Phase V. The object is accepted only if

$$Root'_k = Root_k \quad (77)$$

This verification detects replay, omission, duplication, substitution, or reordering of enclave-generated chunks.

Step 4: Chunk-Key Derivation and Parallel Decryption: For each verified chunk, the user derives the corresponding chunk key

$$K_{chunk}^{(i)} = \text{KDF}(K_{master}, BID, SubID_i, epoch_k) \quad (78)$$

and decrypts

$$\mathcal{M}_i \leftarrow \text{Dec}(K_{chunk}^{(i)}, C_i) \quad (79)$$

Since chunk decryption is independent, the user can decrypt verified chunks in parallel.

Step 5: Deterministic Aggregation Recovery: The decrypted chunks are ordered according to their sub-batch identifiers and concatenated as

$$M_{agg} = \mathcal{M}_{(1)} \parallel \mathcal{M}_{(2)} \parallel \dots \parallel \mathcal{M}_{(r)} \quad (80)$$

Thus, the user recovers the complete aggregated IIoT data only after signature verification, access-policy satisfaction, execution-commitment validation, and chunk-level authenticated decryption succeed.

IV. GROUP-BASED FAILURE DETECTION AND SECURE WORKLOAD RECOVERY

To complement Spider scheduling, this section presents a decentralized failure-detection and recovery mechanism. Instead of requiring every fog node to continuously report heartbeat messages to the MFN, Spider partitions fog nodes into small monitoring groups. This reduces control-plane overhead, avoids centralized monitoring bottlenecks, and enables fast local failure confirmation.

A. Collaborative Failure Detection

The fog layer is divided into bounded-size monitoring groups

$$G_i = \{F_1, F_2, \dots, F_s\} \quad (81)$$

where s is a small constant. Thus, each node monitors only local peers, and the per-node monitoring overhead remains $O(s)$ instead of growing with the total number of fog nodes.

At each monitoring interval, node F_j broadcasts an authenticated heartbeat

$$HB_j(t) = (ID_j, t, epoch_j, status_j, \sigma_j) \quad (82)$$

where $status_j$ contains summarized runtime information and σ_j protects heartbeat integrity. A peer marks F_j as suspicious if

$$\Delta t_j = t_{current} - t_{last}(F_j) > \tau_h \quad (83)$$

To reduce false positives caused by transient congestion, Spider uses quorum-based confirmation. A node is declared failed only if

$$|\mathcal{V}_j| \geq q, \quad q = \left\lceil \frac{s}{2} \right\rceil \quad (84)$$

where $\mathcal{V}_j$ denotes the set of peers reporting timeout violations for F_j . Once confirmed, the failed node is removed from the active scheduling pool.

B. Secure Recovery from Continuation State

For workloads assigned by Spider, each active sub-batch maintains a lightweight continuation state generated during Phase IV:

$$\Xi_k = (ID_k, SubID_k, Prog_k, \Phi(B_k), A_k, epoch_k, T_k, \sigma_k) \quad (85)$$

Unlike full enclave checkpointing, Ξ_k stores only the minimum metadata required to resume execution, including workload identity, sub-batch identity, execution progress, workload profile, policy reference, epoch, timestamp, and authentication token. Completed sub-batches remain valid, while only unfinished sub-batches are reassigned.

When a failure is confirmed, Spider computes the remaining workload

$$\omega_k^{rem} = (1 - Prog_k)\omega_k \quad (86)$$

and ranks recovery tasks according to

$$R_k = \lambda_1 \omega_k^{rem} + \lambda_2 \eta_k + \lambda_3 \delta_k \quad (87)$$

This prioritizes recovery tasks with high remaining cost, high security urgency, and tight deadlines.

C. Capability-Aware Recovery Scheduling

The recovery node is selected from the active fog-node set using the same Spider scheduling logic:

$$F_{recover} = \arg \min_{F_j \in \mathcal{F}_{alive}} Score_F(F_j, B_k) \quad (88)$$

After reassignment, the recovery node verifies σ_k , checks the freshness of $epoch_k$ and T_k , and resumes execution

from the preserved continuation state. This prevents stale-state replay, unauthorized recovery injection, and unnecessary recomputation.

Through local heartbeat monitoring, quorum-based confirmation, authenticated continuation states, and capability-aware recovery scheduling, Spider provides scalable and secure fault tolerance for heterogeneous fog-assisted IIoT environments.

V. SECURITY ANALYSIS

This section analyzes the security of Spider under a quantum-capable adversarial model. The analysis focuses on confidentiality, bounded scheduler leakage, access-control enforcement, integrity, deterministic aggregation correctness, recovery-state authenticity, and post-quantum resistance.

A. Security Goals and Leakage Model

Spider aims to achieve the following security properties: (i) confidentiality of outsourced IIoT data (ii) policy privacy up to structural leakage (iii) bounded scheduler leakage (iv) integrity and authenticity of packets, chunks, and outsourced objects (v) deterministic aggregation correctness (vi) post-quantum resistance

We consider a quantum polynomial-time adversary $\mathcal{A}$ that can observe, modify, replay, delay, or drop public messages. The cloud is honest-but-curious, REEs are untrusted, and TEEs are trusted to protect enclave-resident code, memory, sealed keys, and authentication tokens. The AA is trusted to correctly generate system parameters and attribute keys.

For epoch e , the outsourced object is

$$\Omega_e = (BID_e, epoch_e, CT_e^{AES}, CT_e^{ABE}, Root_e, \sigma_e) \quad (89)$$

Spider exposes only the leakage profile

$$\mathcal{L} = (L_{\text{setup}}, L_{\text{enc}}, L_{\text{sched}}, L_{\text{ret}}) \quad (90)$$

where L_{setup} contains public parameters, L_{enc} contains ciphertext size, batch size, chunk count, and coarse timing information, L_{sched} contains $(\Phi(B_e), F_e^*, r_e)$, and L_{ret} contains retrieval epoch and policy-satisfaction result. No plaintext, session key, aggregation state, chunk key, or enclave-resident secret is included in $\mathcal{L}$

B. Bounded Scheduler Leakage

Lemma 1 (Bounded Scheduler Leakage). *The scheduling information visible to Spider*

$$L_{\text{sched}}(B_e) = (\Phi(B_e), F_e^*, r_e)$$

reveals only coarse workload and placement information and does not reveal plaintext contents, semantic attributes, aggregation states, session keys, chunk keys, or enclave secrets.

Proof. The workload profile $\Phi(B_e)$ is a many-to-one abstraction of the validated batch and contains only batch size, workload intensity, security urgency, and deadline sensitivity. The selected fog node F_e^* and decomposition factor r_e are computed from $\Phi(B_e)$ and runtime resource states, not from plaintext contents. Since all plaintext processing, key derivation, and aggregation states remain inside TEEs, the scheduler

observes no secret-dependent values. Therefore, scheduler leakage is bounded by L_{sched} $\square$

C. Confidentiality of Outsourced Data

Theorem 1 (Leakage-Aware Confidentiality). *Assume that Kyber is IND-CCA secure, the KDF is a quantum-secure PRF, ChaCha20-Poly1305 and AES-GCM are AEAD-secure, lattice-based CP-ABE is selectively secure under MLWE, and TEEs preserve enclave confidentiality. Then Spider reveals no information about IIoT plaintexts beyond $\mathcal{L}$.*

Proof. We prove confidentiality through a sequence of hybrid games.

Game G_0 : The real execution of Spider.

Game G_1 : Replace all device session keys

$$K_{S,i} = \text{KDF}(K_{\text{base},i} \parallel t_i \parallel \text{ctr}_i)$$

with uniformly random strings. By Kyber IND-CCA security and KDF pseudorandomness,

$$|\Pr[G_0] - \Pr[G_1]| \leq \text{Adv}_{\text{Kyber}}^{\text{IND-CCA}} + \text{Adv}_{\text{KDF}}^{\text{PRF}} \quad (91)$$

Game G_2 : Replace all ChaCha20-Poly1305 device ciphertexts and AES-GCM aggregation chunks with encryptions of zero messages of equal length. Then,

$$|\Pr[G_1] - \Pr[G_2]| \leq \text{Adv}_{\text{ChaCha20}}^{\text{AEAD}} + \text{Adv}_{\text{AES}}^{\text{AEAD}} \quad (92)$$

Game G_3 : Replace the CP-ABE ciphertext protecting K_{master} with a simulated ciphertext generated from policy structure alone. By selective security of lattice-based CP-ABE,

$$|\Pr[G_2] - \Pr[G_3]| \leq \text{Adv}_{\text{CP-ABE}}^{\text{SEL-CPA}} \quad (93)$$

Since plaintext processing, aggregation states, and key derivation are confined within TEEs, the adversary observes only

$$\mathcal{L} = (L_{\text{setup}}, L_{\text{enc}}, L_{\text{sched}}, L_{\text{ret}})$$

Thus G_3 is computationally indistinguishable from an ideal simulator that generates outputs solely from $\mathcal{L}$. Therefore,

$$\begin{aligned} \text{Adv}_{\Pi, \mathcal{A}}^{\text{conf}} &\leq \text{Adv}_{\text{Kyber}}^{\text{IND-CCA}} + \text{Adv}_{\text{KDF}}^{\text{PRF}} \\ &\quad + \text{Adv}_{\text{ChaCha20}}^{\text{AEAD}} + \text{Adv}_{\text{AES}}^{\text{AEAD}} \\ &\quad + \text{Adv}_{\text{CP-ABE}}^{\text{SEL-CPA}} + \text{negl}(\lambda) \end{aligned} \quad (94)$$

Since all terms are negligible, Spider reveals no information beyond the prescribed leakage profile $\mathcal{L}$ $\square$

D. Integrity and Authenticity

Theorem 2 (End-to-End Integrity and Authenticity). *Assume HMAC is unforgeable, ChaCha20-Poly1305 and AES-GCM provide ciphertext integrity, Dilithium is EUF-CMA secure, and the hash function is collision resistant. Then forged packets, modified chunks, manipulated commitments, or forged outsourced objects are accepted only with negligible probability.*

Proof. Spider enforces integrity and authenticity through four sequential protection layers.

(1) Device Packet Authenticity Each packet contains an authentication tag

$$Auth_i = \text{HMAC}_{K_{S,i}}(ID_i \parallel Metadata_i \parallel CT_i \parallel Tag_i) \quad (95)$$

where $K_{S,i}$ is derived from the hardware-bound session key. Therefore, any adversary that generates a valid forged packet without knowledge of $K_{S,i}$ breaks HMAC unforgeability.

(2) Enclave Chunk Integrity Each aggregation chunk is protected by AES-GCM:

$$(C_i, Tag_i) \leftarrow \text{AES-GCM.Enc}(K_{chunk}^{(i)}, M_i) \quad (96)$$

Any successful modification of C_i or Tag_i that passes verification implies breaking the INT-CTXT security of AES-GCM.

(3) Aggregation Commitment Integrity Spider binds all enclave outputs using

$$Root_e = H(BID_e \parallel epoch_e \parallel h_{(1)} \parallel \dots \parallel h_{(r)}) \quad (97)$$

where $h_{(i)}$ denotes the digest of the i -th enclave output. Because the digest sequence is deterministically ordered, any replay, duplication, omission, substitution, or reordering attack changes at least one digest or its position, producing

$$Root'_e \neq Root_e \quad (98)$$

except with negligible probability under the collision resistance of $H(\cdot)$

(4) Fog Output Authenticity The final outsourced object is signed using Dilithium:

$$\sigma_e = \text{Sign}_{sk_{FN}}(H(CT_e^{AES} \parallel CT_e^{ABE} \parallel Root_e)) \quad (99)$$

A forged outsourced object accepted by the verifier implies a successful EUF-CMA attack against Dilithium.

Combining all layers, the adversarial success probability satisfies

$$Adv_{\Pi, \mathcal{A}}^{\text{forge}} \leq Adv_{\text{HMAC}}^{\text{UF}} + Adv_{\text{AES-GCM}}^{\text{INT-CTXT}} + Adv_H^{\text{CR}} + Adv_{\text{Dilithium}}^{\text{EUF-CMA}} + \text{negl}(\lambda) \quad (100)$$

Since all terms are negligible under the stated assumptions, forged packets, modified chunks, manipulated commitments, and forged outsourced objects are accepted only with negligible probability. Therefore, Spider achieves end-to-end integrity and authenticity. $\square$

E. Deterministic Aggregation Correctness

Theorem 3 (Deterministic Aggregation Correctness). *If the execution commitment verifies, then the reconstructed aggregation contains every valid enclave sub-batch exactly once and in the deterministic order defined by Spider.*

Proof. Let the validated batch be

$$B_e = \{B_e^{(1)}, B_e^{(2)}, \dots, B_e^{(r)}\} \quad (101)$$

where Spider performs enclave-parallel decomposition such that

$$\bigcup_{i=1}^r B_e^{(i)} = B_e \quad (102)$$

and

$$B_e^{(i)} \cap B_e^{(j)} = \emptyset, \quad i \neq j \quad (103)$$

Thus, every packet belongs to exactly one enclave sub-batch.

Each sub-batch is assigned a unique identifier $SubID_i$ and produces an authenticated digest

$$h_i = H(BID_e \parallel SubID_i \parallel C_i \parallel epoch_e) \quad (104)$$

Spider constructs the execution commitment

$$Root_e = H(BID_e \parallel epoch_e \parallel h_{(1)} \parallel \dots \parallel h_{(r)}) \quad (105)$$

where $(h_{(1)}, \dots, h_{(r)})$ denotes the digest sequence ordered according to increasing $SubID$ values.

During retrieval, the user recomputes

$$Root'_e = H(BID_e \parallel epoch_e \parallel h'_{(1)} \parallel \dots \parallel h'_{(r)}) \quad (106)$$

and accepts only if

$$Root'_e = Root_e \quad (107)$$

Any omission, duplication, substitution, or reordering of a sub-batch changes either a digest value or its position in the ordered sequence, yielding

$$Root'_e \neq Root_e \quad (108)$$

except with negligible probability under the collision resistance of $H(\cdot)$

Therefore, acceptance of the commitment implies that every valid sub-batch is included exactly once and merged according to the unique deterministic ordering defined by Spider. Hence, the reconstructed aggregation is complete, unique, and identical to the aggregation generated by the enclave-parallel execution process. $\square$

F. Policy Privacy and Access-Control Enforcement

Theorem 4 (Policy Privacy and Access-Control Enforcement). *Assuming collision resistance of $\mathcal{H}$ and selective security of the lattice-based CP-ABE scheme, Spider reveals only policy dimensions and structural information, while only users whose attributes satisfy the access policy can recover the master aggregation key.*

Proof. For each attribute $attr_j$, Spider computes

$$ID_j = \mathcal{H}(attr_j) \quad (109)$$

and constructs the access structure using only the hashed identifiers. Under the collision resistance of $\mathcal{H}$, an adversary cannot efficiently recover attribute semantics from the observed identifiers except with negligible probability.

Let the access policy be represented by the LSSS structure (M, ρ) embedded in the CP-ABE ciphertext. The adversary may observe the policy size and structure, denoted by

$$L_{\text{policy}} = (\ell(M), \tau(M)) \quad (110)$$

where $\ell(M)$ and $\tau(M)$ represent the dimensions and structural form of the access matrix. No semantic attribute labels are revealed.

For access-control enforcement, let $Attr_u$ denote the attribute set possessed by user u . Decryption succeeds only if there exists a set of reconstruction coefficients $\{\omega_i\}_{i \in I}$ such that

$$\sum_{i \in I} \omega_i M_i = (1, 0, \dots, 0) \quad (111)$$

where I corresponds to rows whose attributes satisfy the access policy. In this case, the user can reconstruct the protected secret and recover the master aggregation key.

Conversely, if $Attr_u$ does not satisfy the access structure, no valid reconstruction coefficients exist. By the selective-security property of the lattice-based CP-ABE scheme,

$$Adv_{CP-ABE}^{\text{unauth}}(\lambda) \leq \text{negl}(\lambda) \quad (112)$$

implying that an unauthorized adversary cannot recover the master aggregation key with non-negligible probability.

Therefore, Spider reveals only policy structural information while ensuring that only users satisfying the access policy can decrypt the protected aggregation key and access the outsourced IIoT data. $\square$

G. Secure Delegation and Recovery Authenticity

Theorem 5 (Recovery-State Authenticity). *Assume Dilithium or the employed signature scheme is EUF-CMA secure and Kyber is IND-CCA secure. Then an adversary cannot forge, replay, or tamper with a valid continuation state except with negligible probability.*

Proof. For each delegated workload, Spider constructs a continuation state Ξ_k and authenticates it as

$$\sigma_k = \text{Sign}_{SK_{FN}}(H(\Xi_k)) \quad (113)$$

The delegated state is encrypted under a Kyber-derived session key:

$$(K_s, CT_{KEM}) \leftarrow \text{Encaps}(pk_{FN_r}) \quad (114)$$

$$CT_{del} = \text{Enc}_{K_s}(\Xi_k \parallel \sigma_k) \quad (115)$$

A recovery node accepts the delegated state only if

$$\text{Verify}_{PK_{FN}}(\sigma_k, H(\Xi_k)) = 1 \quad (116)$$

and

$$epoch_k = epoch_{current} \quad \wedge \quad T_k < T_{exp}. \quad (117)$$

Thus, forging a valid recovery state requires either producing a valid signature on a new Ξ_k or modifying Ξ_k without changing $H(\Xi_k)$. The former contradicts EUF-CMA security, while the latter contradicts collision resistance of $H(\cdot)$. Replayed states are rejected by the epoch and timestamp checks. Moreover, the confidentiality of Ξ_k is reduced to the IND-CCA security of Kyber.

Therefore,

$$Adv_{II,A}^{\text{rec}} \leq Adv_{\text{Sig}}^{\text{EUF-CMA}} + Adv_H^{\text{CR}} + Adv_{\text{Kyber}}^{\text{IND-CCA}} + \text{negl}(\lambda) \quad (118)$$

Since all terms are negligible, Spider preserves recovery-state authenticity, freshness, and confidentiality. $\square$

H. Forward Secrecy of Session and Chunk Keys

Theorem 6 (Forward Secrecy). *Assume Kyber is IND-CCA secure and the KDF is pseudorandom. Compromise of later protocol states does not reveal previous device session keys or enclave chunk keys except with negligible probability.*

Proof. Device session keys are derived as

$$K_{S,i} = \text{KDF}(K_{base,i} \parallel t_i \parallel ctr_i) \quad (119)$$

where $K_{base,i}$ is obtained from the Kyber shared secret and the PUF-derived device secret. Similarly, chunk keys are derived as

$$K_{chunk}^{(i)} = \text{KDF}(K_{master} \parallel BID \parallel SubID_i \parallel epoch_k) \quad (120)$$

The inclusion of (t_i, ctr_i) and $(BID, SubID_i, epoch_k)$ ensures domain separation across sessions, chunks, and epochs. Therefore, for any two distinct keys K_a and K_b

$$K_a \neq K_b \implies \Pr[K_a \leftrightarrow K_b] \leq Adv_{\text{KDF}}^{\text{PRF}} \quad (121)$$

Consider a completed session i^* . In the security experiment, replace the underlying Kyber secret used in K_{base,i^*} with a uniform random value. By IND-CCA security

$$|\Pr[G_0] - \Pr[G_1]| \leq Adv_{\text{Kyber}}^{\text{IND-CCA}} \quad (122)$$

Next, replace the KDF output with a uniformly random string. Then,

$$|\Pr[G_1] - \Pr[G_2]| \leq Adv_{\text{KDF}}^{\text{PRF}} \quad (123)$$

In G_2 , the challenged session key is independent of all later protocol states, recovered chunk keys, or future epochs. Hence,

$$\Pr[G_2 = 1] = \frac{1}{2} \quad (124)$$

Therefore,

$$Adv_{II,A}^{\text{FS}} \leq Adv_{\text{Kyber}}^{\text{IND-CCA}} + Adv_{\text{KDF}}^{\text{PRF}} + \text{negl}(\lambda) \quad (125)$$

Since both advantages are negligible, compromise of later protocol states does not reveal previously established device session keys or enclave chunk keys. Thus, Spider achieves forward secrecy. $\square$

VI. EVALUATION

This section evaluates the effectiveness, scalability, security-processing efficiency, and fault-tolerance capabilities of Spider in heterogeneous fog-assisted IIoT environments. The evaluation consists of four parts: (i) theoretical computation-cost analysis, (ii) performance evaluation of Spider's hierarchical scheduling and enclave-aware execution mechanisms, and (iii) cryptographic processing overhead. Together, these experiments demonstrate the benefits of integrating post-quantum security, TEE-assisted processing, enclave-aware scheduling, and secure workload recovery within a unified framework.

TABLE III
NOTATION USED IN COMPLEXITY ANALYSIS

Notation	Description
$ F $	Number of fog nodes
m_j	Enclaves in fog node F_j
$\sum_j m_j$	Total enclaves in the fog layer
T	Number of workload requests
P	Number of processing tasks
$ MC $	Number of participating centers
R	Number of regions
I	Number of learning iterations
d	Average neighbor degree
N_{usage}	Network traffic volume
s	Monitoring-group size

A. Theoretical Complexity Analysis

To evaluate the scalability of Spider, we analyze the computational and communication overhead associated with hierarchical scheduling, enclave orchestration, and fault recovery. Unlike existing approaches that focus solely on fog resource allocation, Spider jointly considers fog-node selection, enclave scheduling, secure workload delegation, and recovery coordination in TEE-enabled heterogeneous fog environments.

Table III summarizes the notation used in the complexity analysis, while Table IV compares the scheduling, communication, load-balancing decision, and recovery complexities of representative fog-computing approaches.

As shown in Table IV, existing schemes primarily focus on workload scheduling and resource allocation without integrated fault recovery. Scheme [22] performs load-aware task assignment with scheduling complexity $O(T|F|)$ and load-balancing decision cost $O(|F|)$. Scheme [37] employs SDN-assisted coordination and server-placement optimization with complexity $O(|MC|^2 + |MC| + P)$. Scheme [39] adopts distributed reinforcement learning, incurring overall scheduling complexity $O(RI|F|T)$, while each online decision requires only $O(|A|)$ action selection.

In contrast, Spider employs hierarchical scheduling across both fog nodes and secure enclaves. For each workload, Spider first selects a suitable fog node and subsequently assigns it to an appropriate enclave based on resource availability and queue status. Consequently, both the scheduling complexity and load-balancing decision cost are bounded by $O(|F| + \sum_j m_j)$, where $|F|$ denotes the number of fog nodes and $\sum_j m_j$ denotes the total number of available enclaves. Although Spider incurs slightly higher decision overhead than simple fog-level schedulers, it simultaneously enables enclave-aware placement, secure execution, and recovery readiness within a single scheduling process.

Unlike the baseline schemes, Spider further integrates decentralized failure detection and recovery. Heartbeat monitoring is confined to bounded-size monitoring groups, resulting in detection complexity $O(s)$, where s is independent of the overall system scale. Upon failure, Spider reuses the same hierarchical scheduling procedure to reconstruct workload placement, yielding recovery complexity $O(|F| + \sum_j m_j)$. The communication complexity follows the same bound because

Spider exchanges only lightweight telemetry, heartbeat, and recovery metadata rather than maintaining global optimization states. Therefore, Spider achieves scalable scheduling, efficient enclave utilization, and low-overhead fault recovery in heterogeneous fog-assisted IIoT environments.

B. Performance Analysis

We conducted a comprehensive experimental evaluation to assess the effectiveness, scalability, and efficiency of Spider in heterogeneous fog-assisted IIoT environments. The evaluation focuses on four key aspects: (i) hierarchical scheduling performance, (ii) enclave-parallel secure processing, (iii) fault-tolerant workload recovery, and (iv) cryptographic overhead.

Experimental Setup: The experiments were performed on a workstation equipped with an Intel Core i5-10600 processor operating at 3.30 GHz and 16 GB RAM, with 11 GB allocated to the experimental environment. Fog nodes were deployed as VirtualBox virtual machines to emulate a distributed fog infrastructure, while Trusted Execution Environments (TEEs) were emulated using QEMU to provide enclave isolation and protected execution.

Spider was implemented using CRYSTALS-Kyber for post-quantum key establishment, ChaCha20-Poly1305 for device-side authenticated encryption, AES-GCM for fog-side aggregation protection, and lattice-based CP-ABE for fine-grained access control. Unless otherwise specified, the reported results represent the average of ten independent executions.

The evaluation is organized into three categories. First, we evaluate Spider's hierarchical scheduling mechanisms, including MFN coordination, inter-node workload distribution, and enclave-aware scheduling. Second, we examine the benefits of enclave-parallel execution and secure workload recovery under dynamic fog conditions. Finally, we analyze the cryptographic processing overhead associated with the proposed post-quantum security architecture.

Experiment 1: Inter-Node Scheduling under Homogeneous and Heterogeneous Fog Nodes: This experiment evaluates the effectiveness of Spider's inter-node scheduling mechanism under both homogeneous and heterogeneous fog environments. The objective is to investigate how capability-aware, trust-aware, and enclave-aware scheduling influences task-completion latency when secure post-quantum workloads are distributed across multiple fog nodes.

The workload is fixed at 2,000 secure processing tasks while the number of fog nodes varies from 2 to 12. Spider is compared with three representative fog scheduling schemes [22], [37], [39].

In the *homogeneous* setting, all fog nodes are configured with identical processing capability, memory resources, and network latency. However, runtime conditions may still differ due to dynamic queue occupancy, enclave contention, EPC pressure, and the availability of reusable cryptographic contexts. This setting evaluates whether Spider can exploit secure-execution awareness even when hardware resources are uniform.

TABLE IV
SCHEDULING AND RECOVERY COMPLEXITY COMPARISON

Scheme	Scheduling Cost	Communication Cost	LB Decision Cost	Failure Detection	Recovery Scheduling
[22]	$O(T F)$	$O(N_{\text{usage}})$	$O(F)$	–	–
[37]	$O(MC ^2 + MC + P)$	$O(Pd)$	$O(MC ^2 + MC + P)$	–	–
[39]	$O(RI F T)$	$O(RId)$	$O(A)$ per decision	–	–
Spider	$O(F + \sum_j m_j)$	$O(F + \sum_j m_j)$	$O(F + \sum_j m_j)$	$O(s)$	$O(F + \sum_j m_j)$

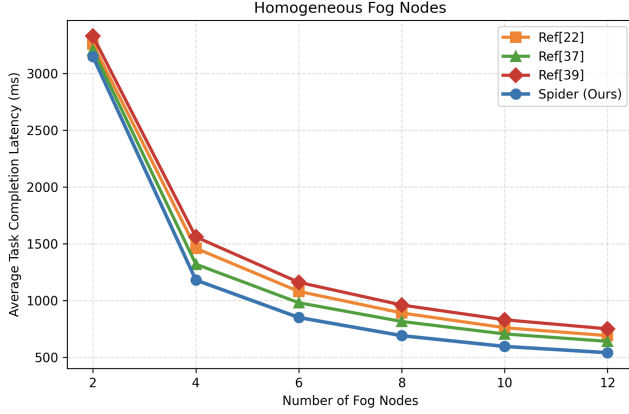


Fig. 2. Task-completion latency under homogeneous fog nodes.

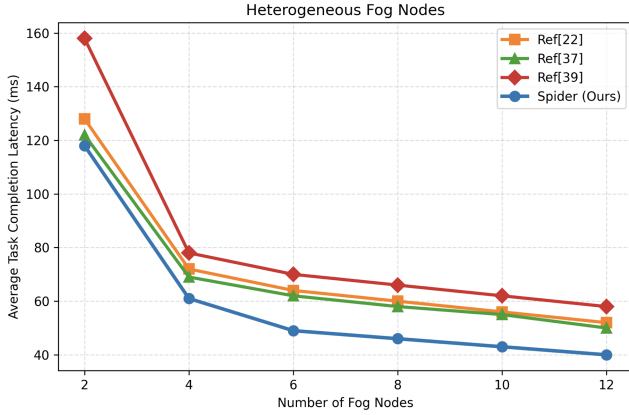


Fig. 3. Task-completion latency under heterogeneous fog nodes.

In the *heterogeneous* setting, fog nodes are assigned different processing capacities, service rates, EPC resources, and communication latencies to emulate realistic large-scale fog deployments.

Fig. 2 shows that task-completion latency decreases as additional fog nodes become available because the workload can be distributed across a larger pool of execution resources. Although all nodes possess similar hardware capability, Spider consistently achieves the lowest latency. The improvement arises from Spider’s hierarchical scheduling strategy, which jointly considers enclave queue occupancy, EPC availability, workload affinity, and cryptographic-context reuse. Consequently, secure workloads are distributed more uniformly across available enclaves, reducing transient resource con-

tention and execution bottlenecks. At 12 fog nodes, Spider reduces task-completion latency by approximately 22%, 16%, and 28% compared with Refs. [22], [37], and [39], respectively.

Fig. 3 further demonstrates the effectiveness of Spider under heterogeneous fog environments. As the number of fog nodes increases, task-completion latency decreases because workloads can be distributed across a larger and more diverse resource pool. Spider consistently achieves the lowest latency across all deployment sizes. At 12 fog nodes, Spider reduces task-completion latency by approximately 23%, 20%, and 31% compared with Refs. [22], [37], and [39], respectively.

The improvement stems from Spider’s capability-aware scoring mechanism, which jointly considers processing capability, queue state, EPC pressure, runtime trust, and communication latency. By avoiding resource-constrained or heavily loaded nodes, Spider achieves more balanced workload distribution and higher resource utilization. Unlike conventional schedulers that primarily consider node-level resource availability, Spider additionally incorporates enclave-level execution dynamics, enabling more efficient scheduling of secure post-quantum workloads.

Experiment 2: Intra-Node Scheduling under Enclave Heterogeneity: This experiment evaluates Spider’s enclave-level scheduling mechanism within a single fog node. The objective is to investigate whether enclave-aware scheduling can improve secure-workload execution when multiple enclaves exhibit different processing capabilities.

A fog node containing four TEE enclaves is considered. The workload is fixed at 2,000 secure processing tasks. To emulate enclave heterogeneity, enclave service rates are varied according to a speed spread ratio

$$\rho = \frac{\max(\mu_i)}{\min(\mu_i)}$$

where μ_i denotes the processing rate of enclave E_i . A larger ρ indicates greater performance disparity among enclaves.

Spider is compared against Round-Robin (RR) scheduling and Least-Queue (LQ) scheduling. The measured metric is the total workload completion time required to process all tasks.

Fig. 4 illustrates the impact of enclave heterogeneity on scheduling performance. When all enclaves exhibit similar processing capability ($\rho = 1$), all schemes achieve nearly identical performance because workload assignment decisions have minimal impact.

As enclave heterogeneity increases, the performance gap becomes more pronounced. Round-Robin and Least-Queue

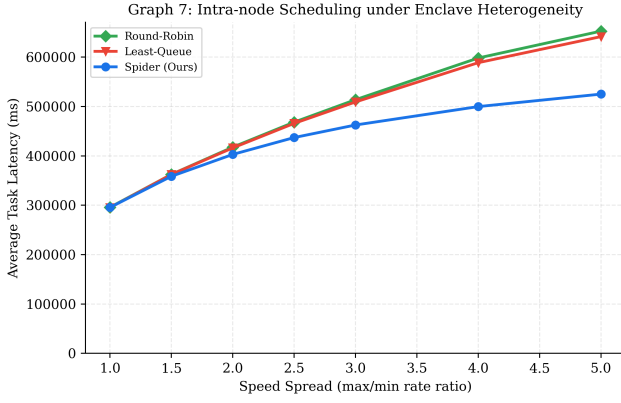


Fig. 4. Workload completion time under enclave heterogeneity.

scheduling continue assigning tasks without fully considering enclave processing capability, resulting in workload accumulation at slower enclaves and increased completion time.

In contrast, Spider jointly considers queue occupancy, enclave service rate, EPC availability, and workload affinity during enclave selection. Consequently, workloads are preferentially assigned to better-performing enclaves while avoiding EPC contention and execution bottlenecks.

At $\rho = 5$, Spider reduces workload completion time by approximately 19% and 18% compared with Round-Robin and Least-Queue scheduling, respectively. The performance improvement increases as enclave heterogeneity grows, demonstrating the effectiveness of Spider’s enclave-aware scheduling mechanism.

Experiment 3: Recovery Latency under Fog-Node Failures: This experiment evaluates the efficiency of Spider’s secure recovery mechanism under fog-node failures. The objective is to measure how quickly unfinished workloads can be recovered and reassigned after a node failure occurs.

The number of fog nodes is varied from 10 to 50. Spider is compared with Centralized Heartbeat (Centralized HB), Full Checkpoint, Round-Robin Recovery, and Least-Queue Recovery. Recovery latency is measured as the elapsed time required to detect a failed node, select a recovery node, restore the preserved workload state, and resume execution.

Fig. 5 shows that Spider consistently achieves the lowest recovery latency across all deployment sizes. While the baseline schemes exhibit larger latency fluctuations as the number of fog nodes increases, Spider remains stable at approximately 32–36 ms. This stability results from decentralized group-based failure detection and lightweight continuation-state recovery.

Centralized HB suffers from higher recovery latency because failure monitoring and recovery coordination depend on a central coordinator. Full Checkpoint incurs additional overhead due to checkpoint restoration. Round-Robin and Least-Queue recovery may select unsuitable recovery nodes because they do not jointly consider node capability, EPC availability, trust condition, and workload continuation state.

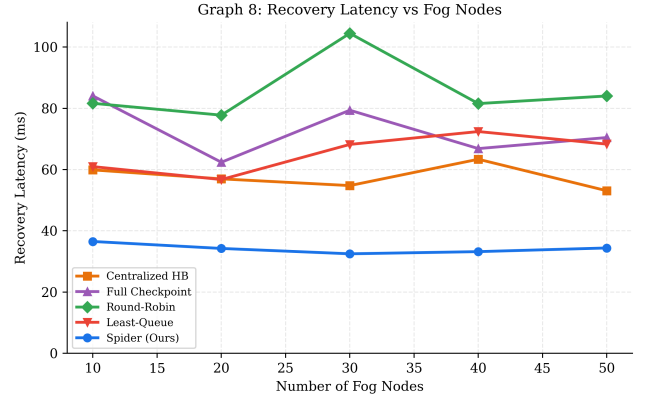


Fig. 5. Recovery latency under fog-node failures.

In contrast, Spider detects failures through local monitoring groups and reassigns only unfinished workload fragments using authenticated continuation states. It also reuses its capability-aware scheduling score to select suitable recovery nodes. As a result, Spider avoids full workload restart, reduces recovery coordination overhead, and maintains low recovery latency even as the fog infrastructure scales.

Experiment 4: Recovery Latency under Fog-Node Failures: This experiment evaluates the efficiency of Spider’s secure workload recovery mechanism when fog-node failures occur during execution. The objective is to measure the time required to recover unfinished workloads and resume processing after a failure.

The workload is fixed at 1,000 secure processing tasks while the number of fog nodes varies from 10 to 50. A failure is randomly injected into one active fog node during execution. Spider is compared with Full Re-execution (FR) and Checkpoint Recovery (CR). Recovery latency is measured as the elapsed time between failure detection and successful workload completion.

Fig. 5 shows that recovery latency increases with the number of fog nodes due to additional coordination and reassignment overhead. Spider consistently achieves the lowest recovery latency across all deployment sizes.

The improvement is achieved because Spider preserves only lightweight authenticated continuation states and recovers only unfinished workloads. In contrast, Full Re-execution restarts all affected tasks, while Checkpoint Recovery incurs additional checkpoint-management and restoration overhead.

Furthermore, Spider reuses its capability-aware scheduling mechanism during recovery, enabling efficient reassignment of unfinished workloads to suitable fog nodes. As a result, Spider significantly reduces recovery latency while preserving secure execution continuity under fog-node failures.

Experiment 5: Task Completion Ratio under Increasing Failure Rates: This experiment evaluates the resilience of Spider under progressively increasing fog-node failure rates. The objective is to determine whether Spider can maintain workload completion despite failures occurring during secure processing.

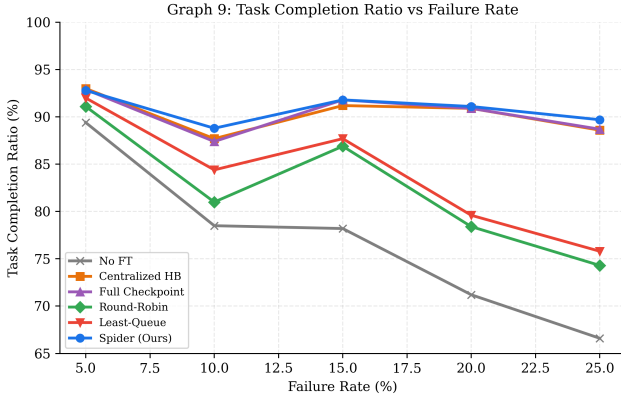


Fig. 6. Task completion ratio under increasing failure rates.

The workload is fixed at 1,000 secure processing tasks and the fog infrastructure consists of 30 fog nodes. The node failure rate is varied from 5 with Full Re-execution (FR) and Checkpoint Recovery (CR). The performance metric is the task completion ratio, defined as the percentage of tasks successfully completed within the specified execution window.

Fig. 6 shows that the task completion ratio decreases for all schemes as the failure rate increases because more workloads are interrupted by node failures. Spider consistently maintains the highest completion ratio across all failure levels. The improvement stems from its group-based failure detection mechanism and lightweight continuation-state recovery strategy, which allow unfinished workloads to be rapidly re-assigned and resumed on healthy fog nodes. In contrast, Full Re-execution suffers the largest reduction in completion ratio because failed workloads must be restarted from the beginning. Checkpoint Recovery partially mitigates this issue but still incurs recovery overhead and checkpoint synchronization delays. As failure rates increase, the advantage of Spider becomes more pronounced because only incomplete workload fragments require recovery, while completed enclave tasks remain unaffected. Consequently, Spider preserves a higher level of service continuity and workload completion under adverse operating conditions.

Experiment 6: Effect of Cache- and Reuse-Aware Scheduling: This experiment evaluates the impact of Spider’s cache- and reuse-aware scheduling mechanism. The objective is to measure whether reusing precomputed policy structures and cryptographic contexts can reduce redundant fog-side processing under increasing workload volume.

The number of tasks is varied from 100 to 2,000 while the fog infrastructure is fixed at five fog nodes. Spider is compared with two baseline strategies: no-cache scheduling and random cache placement. The measured metric is the average encryption latency per task.

Fig. 7 shows that the average encryption latency increases with the number of tasks for all schemes because larger workloads introduce more policy-processing and cryptographic operations. The no-cache baseline generally incurs the high-

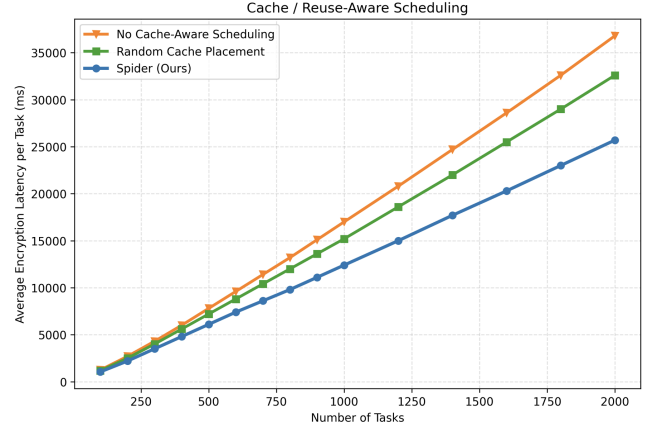


Fig. 7. Average encryption latency under cache- and reuse-aware scheduling.

est latency because each task independently reconstructs the required policy and cryptographic structures. Random cache placement provides limited benefit, as cached contexts are not necessarily aligned with incoming workload characteristics.

Spider achieves slightly lower latency in most workload settings by incorporating workload affinity and cryptographic-context reuse into the scheduling decision. Instead of assigning tasks solely according to resource availability, Spider preferentially routes compatible workloads to fog nodes or enclaves that already maintain reusable policy representations or precomputed lattice components.

Although the improvement is moderate, the result confirms that cache- and reuse-aware scheduling can reduce redundant initialization costs and complement Spider’s hierarchical scheduling mechanism. This optimization is particularly useful when repeated workloads share similar access policies or cryptographic contexts.

Experiment 7: Load-Balancing Decision Quality and Enclave-Aware Scheduling: This experiment evaluates the scalability of Spider’s load-balancing mechanism and the effectiveness of its enclave-aware scheduling strategy through two complementary scenarios.

Scenario 1: Load-Balancing Decision Latency The number of fog nodes is varied from 5 to 50, and the average load-balancing decision latency is measured for Spider and Refs. [22], [37], and [39].

As shown in Fig. 8, decision latency increases with the number of fog nodes because more candidate resources must be evaluated. Ref. [22] achieves the lowest latency due to its lightweight fog-level scheduling strategy. Spider incurs a modest additional overhead because it jointly performs fog-node and enclave selection. Nevertheless, Spider remains significantly more efficient than Ref. [37], which relies on SDN-assisted placement optimization, and Ref. [39], which introduces learning and optimization overhead. These results demonstrate that Spider maintains scalable scheduling despite supporting enclave-aware placement.

Scenario 2: Impact of Enclave-Aware Scheduling To isolate the contribution of enclave awareness, the infras-

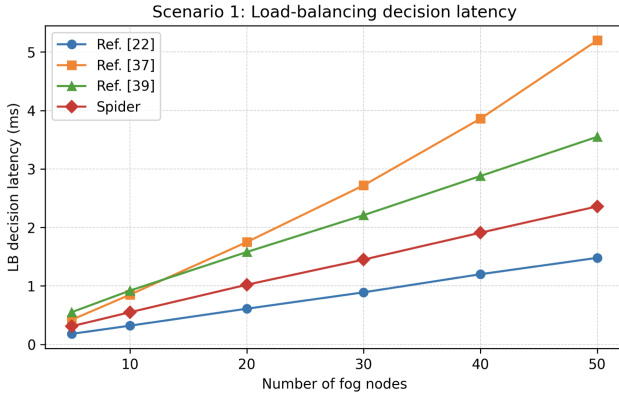


Fig. 8. Load-balancing decision latency under increasing numbers of fog nodes.

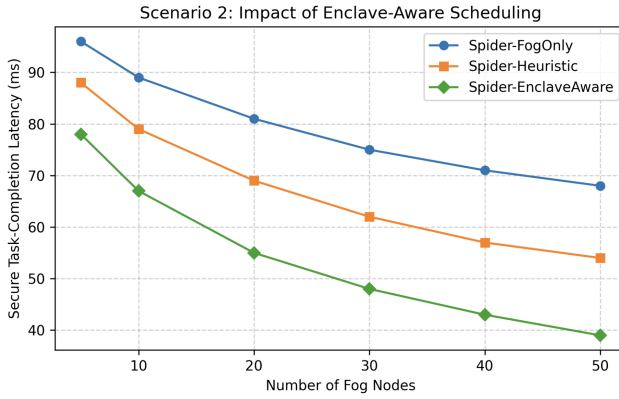


Fig. 9. Secure task-completion latency under increasing enclave contention.

tructure is fixed to 20 fog nodes with four enclaves per node, while the workload intensity is gradually increased to create different levels of enclave contention. Three Spider variants are compared: *Spider-FogOnly*, which uses only fog-node information; *Spider-Heuristic*, which additionally selects the least-loaded enclave; and *Spider-EnclaveAware*, the full scheduler that jointly considers fog-node capability, latency, trust level, enclave queue occupancy, EPC availability, and recovery readiness. Secure task-completion latency is used as the primary performance metric.

Figure 9 shows that *Spider-EnclaveAware* consistently achieves the lowest secure task-completion latency. *Spider-FogOnly* balances workloads only at the fog level and cannot detect enclave-level congestion or EPC pressure, leading to suboptimal placements. *Spider-Heuristic* improves performance by considering enclave queue occupancy, but remains vulnerable to EPC contention. In contrast, *Spider-EnclaveAware* jointly evaluates enclave queue occupancy, EPC availability, trust level, and recovery readiness, thereby avoiding enclave hotspots and reducing queueing delays. As workload intensity increases, the performance gap becomes more pronounced, highlighting the importance of enclave-aware scheduling in TEE-enabled fog environments.

Overall, the results show that Spider introduces only a small increase in scheduling overhead compared with simple fog-level schedulers while providing substantially better workload placement quality. The enclave-aware scheduling mechanism enables efficient utilization of enclave resources, lower secure task-completion latency, and improved resilience under contention, demonstrating a favorable trade-off between scheduling efficiency, security, and fault tolerance.

VII. CONCLUSION

This paper presented *Spider*, a secure and scalable fog-assisted IIoT processing framework that integrates post-quantum protection, TEE-assisted secure execution, hierarchical enclave-aware scheduling, and resilient workload recovery. Spider combines PUF-based device authentication, Kyber key establishment, ChaCha20-Poly1305 data protection, and lattice-based CP-ABE access control to provide end-to-end security for outsourced IIoT data. To efficiently manage security-sensitive workloads, Spider introduces a hierarchical scheduling mechanism that jointly performs fog-node selection, enclave-level workload placement, adaptive batch decomposition, and reuse-aware scheduling while considering capability, trust, queue state, and EPC availability. The framework further incorporates group-based failure detection and secure workload recovery to maintain execution continuity under fog-node failures. Experimental results show that Spider achieves lower scheduling latency, improved load balancing, efficient CP-ABE processing, and faster recovery compared with existing approaches, particularly in heterogeneous fog environments. Future work will investigate adaptive learning-based scheduling, cross-domain fog collaboration, and support for emerging post-quantum cryptographic primitives to further improve scalability, security, and deployment flexibility in large-scale IIoT systems.

REFERENCES

- [1] S. Fugkeaw, A. Changtor, T. Maneerat, P. Rattanasrisuk, and K. Tangtanawirut, "LightPUF-IIoT: A Lightweight PUF-Based Authentication Scheme With Real-Time Detection of Rogue Devices in Fog-Assisted IIoT Data Sharing," *IEEE Open Journal of the Computer Society*, vol. 6, pp. 1438–1450, 2025, doi: 10.1109/OJCS.2025.3607984.
- [2] A. Changtor, T. Maneerat, P. Rattanasrisuk, K. Tangtanawirut, and S. Fugkeaw, "A Secure and Lightweight IIoT Data Authentication Using PUF-Based in IoT Fog-Assisted Cloud," in *Proc. 2025 17th International Conference on Knowledge and Smart Technology (KST)*, 2025, pp. 40–45, doi: 10.1109/KST65016.2025.11003299.
- [3] Y. Yang, J. Zhang, X. Liu, and J. Ma, "A Scalable and Auditable Secure Data Sharing Scheme With Traceability for Fog-Based Smart Logistics," *IEEE Internet of Things Journal*, vol. 10, no. 10, pp. 8603–8617, 2023, doi: 10.1109/JIOT.2022.3220850.
- [4] P. Poomekum, A. Suriyawong, and S. Fugkeaw, "Fine-Grained and Lightweight Quantum-Resistant Access Control System With Efficient Revocation for IoT Cloud," *IEEE Open Journal of the Communications Society*, vol. 6, pp. 8652–8666, 2025, doi: 10.1109/OJCOMS.2025.3620094.
- [5] R. P. Gupta, C. Vorakulpipat, S. Fugkeaw, and A. Witayangkurn, "A Scalable and Secure Outsourced IoT Electronic Health Records with Efficient User Revocation Using Fog-Assisted Cloud Model," in *Proc. 2024 21st International Joint Conference on Computer Science and Software Engineering (JCSSE)*, 2024, pp. 61–67, doi: 10.1109/JCSSE61278.2024.10613723.

- [6] Z. Wang and Y. Zhou, "Analysis and Evaluation of Intel Software Guard Extension-Based Trusted Execution Environment Usage in Edge Intelligence and Internet of Things Scenarios," *Future Internet*, vol. 17, no. 1, Art. no. 32, 2025, doi: 10.3390/fi17010032.
- [7] A. Muñoz, R. Ríos, R. Román, and J. López, "A Survey on the (In)Security of Trusted Execution Environments," *Computers & Security*, vol. 129, Art. no. 103180, 2023, doi: 10.1016/j.cose.2023.103180.
- [8] A. N. Alvi, B. Ali, M. S. Saleh, M. Alkhatami, D. Alsadie, and B. Alghamdi, "Secure Computing for Fog-Enabled Industrial IoT," *Sensors*, vol. 24, no. 7, Art. no. 2098, 2024, doi: 10.3390/s24072098.
- [9] N. Sarwar, R. S. Alharthi, H. M. Mujlid, et al., "Enhancing IoT Communication in Fog Computing: A Lightweight Remote User Authentication and Key Management Scheme Utilizing ECC," *Journal of Cloud Computing*, vol. 14, Art. no. 86, 2025, doi: 10.1186/s13677-025-00816-y.
- [10] W. B. Daoud, S. Othmen, M. Hamdi, et al., "Fog Computing Network Security Based on Resources Management," *Journal of Wireless Communications and Networking*, vol. 2023, Art. no. 50, 2023, doi: 10.1186/s13638-023-02256-1.
- [11] H. Nguyen, S. Huda, Y. Nogami, and T. T. Nguyen, "Security in Post-Quantum Era: A Comprehensive Survey on Lattice-Based Algorithms," *IEEE Access*, vol. 13, pp. 89003–89024, 2025, doi: 10.1109/ACCESS.2025.3571307.
- [12] A. B. Shahid, K. Mansoor, Y. A. Bangash, et al., "Post-Quantum Cryptographic Authentication Protocol for Industrial IoT Using Lattice-Based Cryptography," *Scientific Reports*, vol. 16, Art. no. 9582, 2026, doi: 10.1038/s41598-025-28413-8.
- [13] D. C. Lawo, R. Frantz, A. Cano Aguilera, X. Arnal I Clemente, M. P. Podleš, J. L. Imaña, I. Tafur Monroy, and J. J. Vegas Olmos, "Falcon/Kyber and Dilithium/Kyber Network Stack on Nvidia's Data Processing Unit Platform," *IEEE Access*, vol. 12, pp. 38048–38056, 2024, doi: 10.1109/ACCESS.2024.3374629.
- [14] M. A. González de la Torre, I. A. Morales Sandoval, G. T. Freitas de Abreu, and L. Hernández Encinas, "Post-Quantum Wireless-Based Key Encapsulation Mechanism via CRYSTALS-Kyber for Resource-Constrained Devices," *IEEE Access*, vol. 13, pp. 66714–66725, 2025, doi: 10.1109/ACCESS.2025.3560023.
- [15] L. Cruz-Piris, A. Marín-López, M. Álvarez-Campana, M. Sanz, J. I. Moreno, and D. Arroyo, "Measuring the Impact of Post Quantum Cryptography in Industrial IoT Scenarios," *Internet of Things*, vol. 34, Art. no. 101793, 2025, doi: 10.1016/j.iot.2025.101793.
- [16] S. Aghapour, K. Ahmadi, M. Anastasova, M. Mozaffari Kermani, and R. Azarderakhsh, "PUF-Kyber: Design of a PUF-Based Kyber Architecture Benchmarked on Diverse ARM Processors," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 43, no. 12, pp. 4453–4462, 2024, doi: 10.1109/TCAD.2024.3399669.
- [17] C. Kim, S. Son, and Y. Park, "A Privacy-Preserving Authentication Scheme Using PUF and Biometrics for IoT-Enabled Smart Cities," *Electronics*, vol. 14, no. 10, Art. no. 1953, 2025, doi: 10.3390/electronics14101953.
- [18] C. Rubio García, A. Cano Aguilera, C. Stan, J. J. Vegas Olmos, S. Rommel, and I. Tafur Monroy, "Enhanced Network Security Protocols for the Quantum Era: Combining Classical and Post-Quantum Cryptography, and Quantum Key Distribution," *IEEE Journal on Selected Areas in Communications*, vol. 43, no. 8, pp. 2765–2781, 2025, doi: 10.1109/JSAC.2025.3568011.
- [19] S. Y. Moon, B. H. Jo, A. El Azzaoui, S. K. Singh, and J. H. Park, "Edge-Fog Enhanced Post-Quantum Network Security: Applications, Challenges and Solutions," *Computers, Materials & Continua*, vol. 84, no. 1, pp. 25–55, 2025, doi: 10.32604/cmc.2025.062966.
- [20] D. Aldossary, E. Aldahasi, T. Balharith, and T. Helmy, "A Systematic Literature Review on Load-Balancing Techniques in Fog Computing: Architectures, Strategies, and Emerging Trends," *Computers*, vol. 14, no. 6, Art. no. 217, 2025, doi: 10.3390/computers14060217.
- [21] M. H. Kashani and E. Mahdipour, "Load Balancing Algorithms in Fog Computing," *IEEE Transactions on Services Computing*, vol. 16, no. 2, pp. 1505–1521, 2023, doi: 10.1109/TSC.2022.3174475.
- [22] M. Ala'anzy, R. Zhanuzak, R. Akhmedov, N. Mohamed, and J. Al-Jaroodi, "Dynamic Load Balancing for Enhanced Network Performance in IoT-Enabled Smart Healthcare With Fog Computing," *IEEE Access*, vol. 12, pp. 188957–188975, 2024, doi: 10.1109/ACCESS.2024.3516362.
- [23] S. Fugkeaw, S. Rattagool, P. Jiangthiran, and P. Pholwiset, "FPRESSO: Fast and Privacy-Preserving SSO Authentication With Dynamic Load Balancing for Multi-Cloud-Based Web Applications," *IEEE Access*, vol. 12, pp. 157888–157900, 2024, doi: 10.1109/ACCESS.2024.3485996.
- [24] X. Yu, M. Zhu, M. Zhu, et al., "A Novel Location-Aware Job Scheduling Framework for Optimizing Fog-Cloud IoT Systems: Insights From Dynamic Traffic Management," *Journal of Cloud Computing*, vol. 14, Art. no. 50, 2025, doi: 10.1186/s13677-025-00772-7.
- [25] M. Vijarana, S. Gupta, A. Agrawal, M. O. Adigun, S. A. Ajagbe, and J. B. Awotunde, "Energy Efficient Load-Balancing Mechanism in Integrated IoT-Fog-Cloud Environment," *Electronics*, vol. 12, no. 11, Art. no. 2543, 2023, doi: 10.3390/electronics12112543.
- [26] S. Ijaz, S. G. Ahmad, K. Ayyub, et al., "Energy-Efficient Time and Cost Constraint Scheduling Algorithm Using Improved Multi-Objective Differential Evolution in Fog Computing," *The Journal of Supercomputing*, vol. 81, Art. no. 116, 2025, doi: 10.1007/s11227-024-06550-7.
- [27] J. Wan, B. Chen, S. Wang, M. Xia, D. Li, and C. Liu, "Fog Computing for Energy-Aware Load Balancing and Scheduling in Smart Factory," *IEEE Transactions on Industrial Informatics*, vol. 14, no. 10, pp. 4548–4556, 2018, doi: 10.1109/TII.2018.2818932.
- [28] J. Singh, P. Singh, E. M. Amhoud, and M. Hedabou, "Energy-Efficient and Secure Load Balancing Technique for SDN-Enabled Fog Computing," *Sustainability*, vol. 14, no. 19, Art. no. 12951, 2022, doi: 10.3390/su141912951.
- [29] H. Wang, L. Cai, X. Hao, J. Ren, and Y. Ma, "ETS-TEE: An Energy-Efficient Task Scheduling Strategy in a Mobile Trusted Computing Environment," *Tsinghua Science and Technology*, vol. 28, no. 1, pp. 105–116, 2023, doi: 10.26599/TST.2021.9010088.
- [30] Y. Li, D. Zeng, L. Gu, A. Zhu, Q. Chen, and S. Yu, "PASTO: Enabling Secure and Efficient Task Offloading in TrustZone-Enabled Edge Clouds," *IEEE Transactions on Vehicular Technology*, vol. 72, no. 6, pp. 8234–8238, 2023, doi: 10.1109/TVT.2023.3237204.
- [31] S. Fugkeaw, R. Prasad Gupta and K. Worapaluk, "Secure and Fine-Grained Access Control With Optimized Revocation for Outsourced IoT EHRs With Adaptive Load-Sharing in Fog-Assisted Cloud Environment," in *IEEE Access*, vol. 12, pp. 82753–82768, 2024, doi: 10.1109/ACCESS.2024.3412754.
- [32] Z. Liu et al., "A Privacy-Preserving Outsourcing Computing Scheme Based on Secure Trusted Environment," in *IEEE Transactions on Cloud Computing*, vol. 11, no. 3, pp. 2325–2336, 1 July–Sept. 2023, doi: 10.1109/TCC.2022.3201401.
- [33] Y. Bai, D. He, Z. Yang, M. Luo and C. Peng, "Efficient Module-Lattice-Based Certificateless Online/Offline Signcryption Scheme for Internet of Medical Things," in *IEEE Internet of Things Journal*, vol. 12, no. 14, pp. 27350–27363, 15 July 2025, doi: 10.1109/IIOT.2025.3562262.
- [34] W. Zhang, J. Wei and M. K. Khan, "PQC-Enhanced Privacy-Preserving Federated Learning for Edge-Based IoT Ecosystems," in *IEEE Transactions on Consumer Electronics*, vol. 71, no. 4, pp. 11171–11182, Nov. 2025, doi: 10.1109/TCE.2025.3601395.
- [35] A. N. Zaheer, M. Farhan, M. Rehan Naeem and M. M. Alnfaia, "Quantum-Resilient Cryptographic Frameworks: Design and Analysis of Post-Quantum Algorithms for Secure and Efficient Edge-Assisted IoT Ecosystems in Consumer Electronics Devices," in *IEEE Transactions on Consumer Electronics*, vol. 71, no. 4, pp. 11258–11268, Nov. 2025, doi: 10.1109/TCE.2025.3619556.
- [36] Z. Man, Z. Yu, J. Yu, C. Gao and X. Meng, "Edge Computing in Internet of Things: Lattice-Based and Split Encryption for Post-Quantum Data Security," in *IEEE Internet of Things Journal*, vol. 12, no. 23, pp. 49327–49339, 1 Dec. 1, 2025, doi: 10.1109/IIOT.2025.3591521.
- [37] A. M. Jasim and H. Al-Raweshidy, "An Adaptive SDN-Based Load Balancing Method for Edge/Fog-Based Real-Time Healthcare Systems," in *IEEE Systems Journal*, vol. 18, no. 2, pp. 1139–1150, June 2024, doi: 10.1109/JSYST.2024.3402156.
- [38] R. M. Singh, G. Sikka and L. K. Awasthi, "LBATSM: Load Balancing Aware Task Selection and Migration Approach in Fog Computing Environment," in *IEEE Systems Journal*, vol. 18, no. 2, pp. 796–804, June 2024, doi: 10.1109/JSYST.2024.3403673.
- [39] E. Oustad, "DIST: Distributed Learning-Based Energy-Efficient and Reliable Task Scheduling and Resource Allocation in Fog Computing," in *IEEE Transactions on Services Computing*, vol. 18, no. 3, pp. 1336–1351, May–June 2025, doi: 10.1109/TSC.2025.3568255.
- [40] M. Maashi, "Elevating Survivability in Next-Gen IoT-Fog-Cloud Networks: Scheduling Optimization With the Metaheuristic Mountain Gazelle Algorithm," in *IEEE Transactions on Consumer Electronics*, vol. 70, no. 1, pp. 3802–3809, Feb. 2024, doi: 10.1109/TCE.2024.3371774.

- [41] D. Kang and H. J. Jo, "TB-Logger: Secure Vehicle Data Logging Method Using Trusted Execution Environment and Blockchain," in *IEEE Access*, vol. 11, pp. 23282-23292, 2023, doi: 10.1109/ACCESS.2023.3253626.
- [42] C. Wang, Y. Deng, Z. Ning, K. Leach, J. Li, S. Yan, Z. He, J. Cao, and F. Zhang, "Building a Lightweight Trusted Execution Environment for Arm GPUs," in *IEEE Transactions on Dependable and Secure Computing*, vol. 21, no. 4, pp. 3801-3816, 2024, doi: 10.1109/TDSC.2023.3334277.
- [43] J. Zhao, H. Zhu, F. Wang, R. Lu, and H. Li, "SXGB: Secure and Efficient Vertical Federated XGBoost via Trusted Execution Environments," in *IEEE Transactions on Dependable and Secure Computing*, vol. 23, no. 2, pp. 2275-2288, Mar. 2026, doi: 10.1109/TDSC.2025.3626379.
- [44] L. Hak and S. Fugkeaw, "SSL-XIoMT: Secure, Scalable, and Lightweight Cross-Domain IoMT Sharing With SSI and ZKP Authentication," *IEEE Open Journal of the Computer Society*, vol. 6, pp. 714–725, 2025, doi: 10.1109/OJCS.2025.3570087.